COMP3011
计算机体系结构

1 绪论

授课教师：王强

邮箱：qiang.wang@hit.edu.cn



目录 | CONTENT

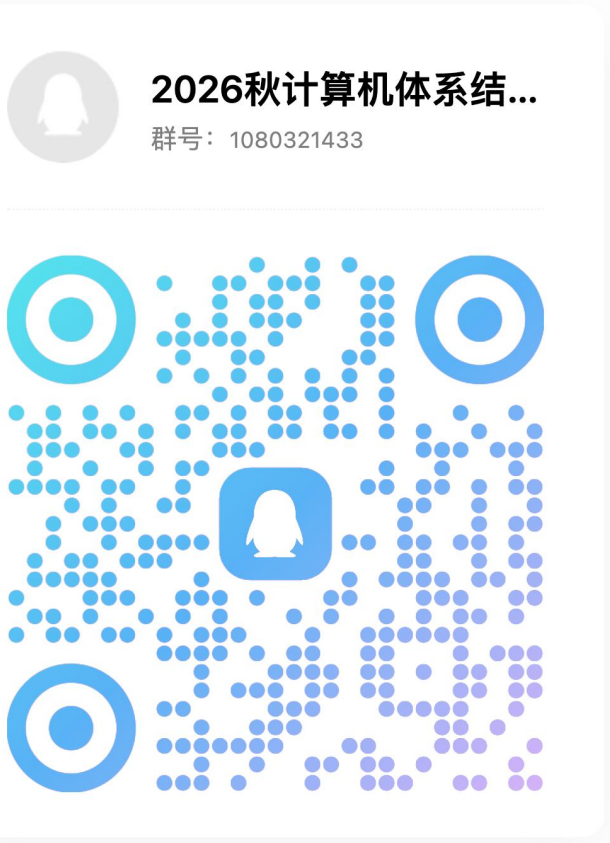
1 课程介绍

2 计算机体系结构的发展历史

3 体系结构面临的挑战和发展机遇

授课老师与助教

- 授课老师
  - 王强
  - 邮箱：qiang.wang@hit.edu.cn
  - 办公室：L1605
  - 主要研究方向
    - GPU计算
    - 高性能计算
    - 算力管理与任务调度



扫一扫二维码，加入群聊

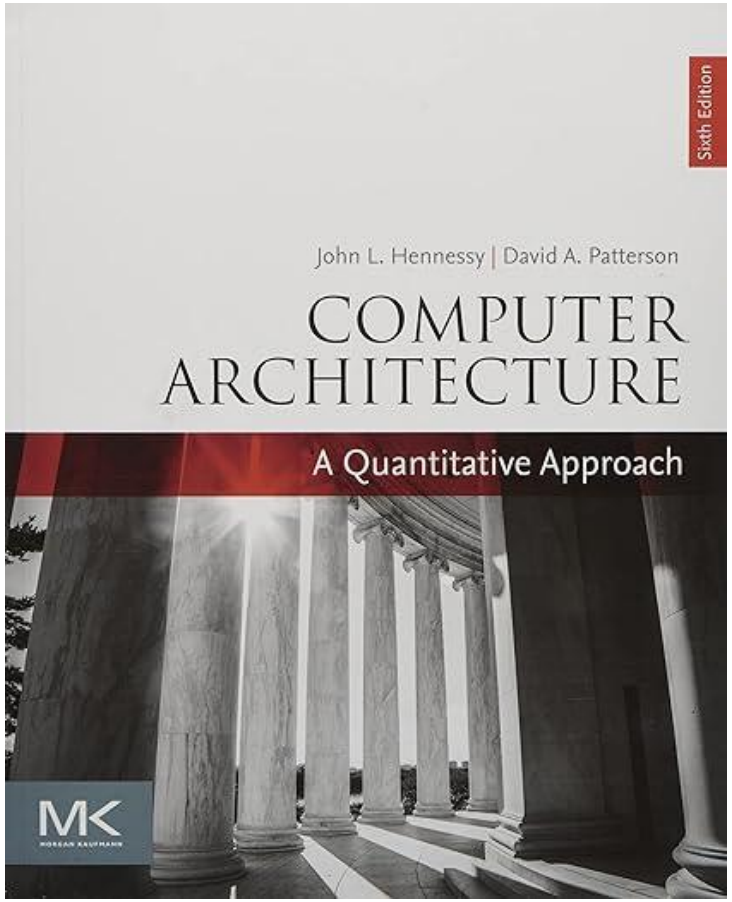
理论课QQ群（群号：1080321433）

课程教材与参考书

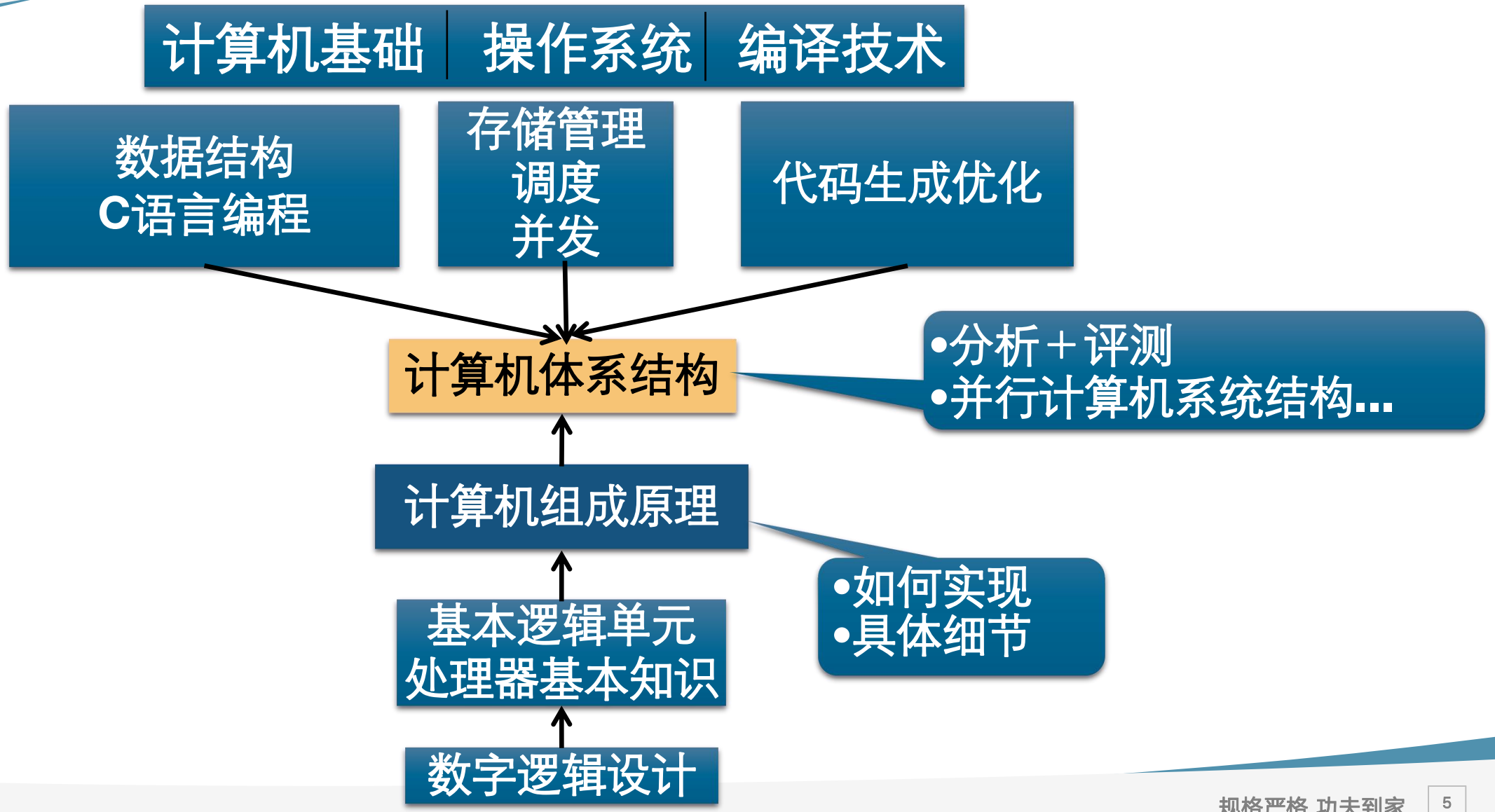
- 教材： John L. Hennessy, David A. Patterson. Computer Architecture: A Quantitative Approach, 6th Edition. Morgan Kaufmann, Sep 2017.
- 主要参考书：
  1. 张晨曦等. 计算机系统结构教程（第3版），清华大学出版社，2021.
  2. 李学干，计算机系统结构（第五版），西安电子科技大学出版社，2011.







本课程与CS其他课程间的关系



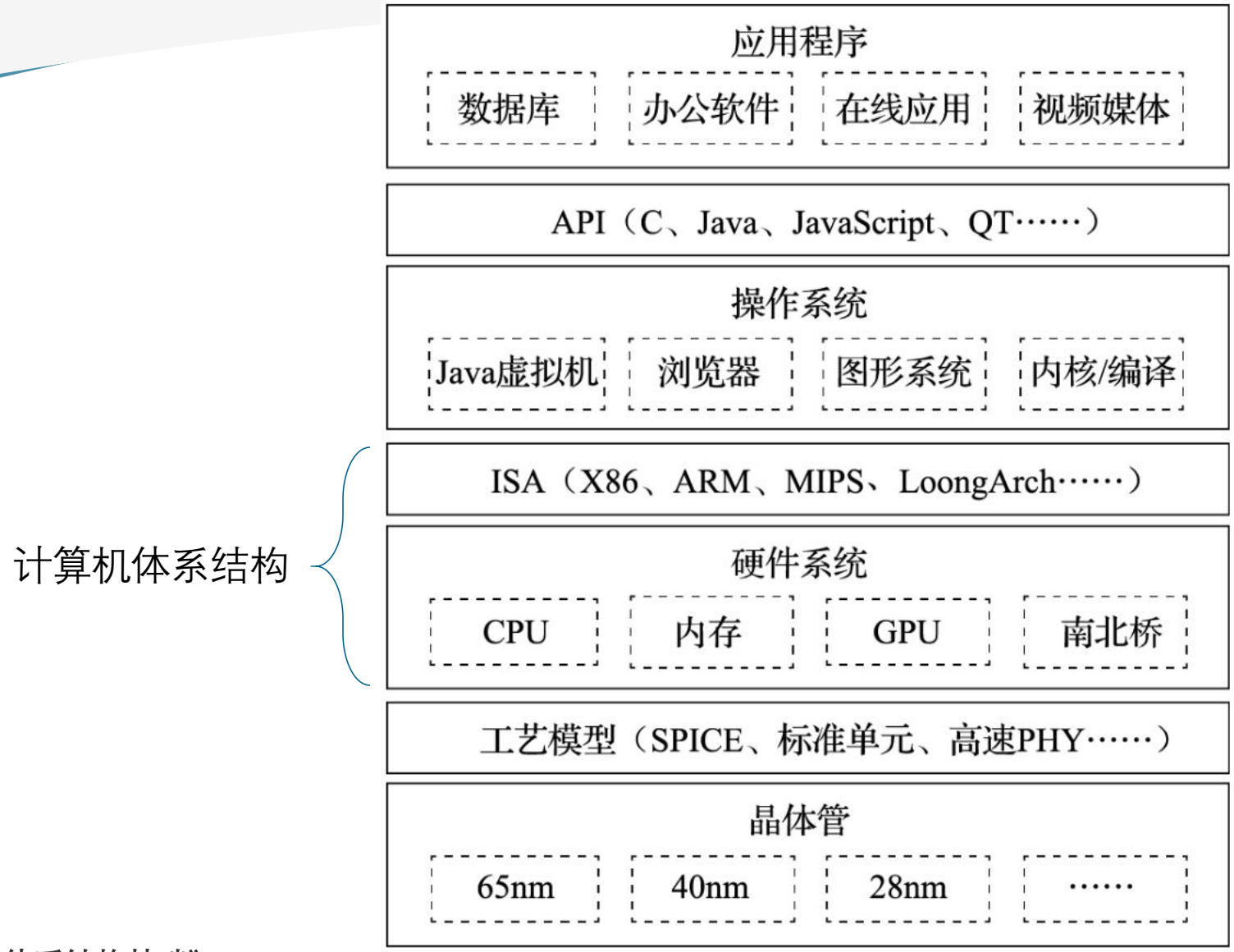
计算机组成原理 VS 计算机体系结构

- **计算机组成（偏重实现）**
  - 实现计算机体系结构所体现的属性
  - **比如具体指令的实现**

- 机器是否具备乘法指令的功能，指令操作的数据类型？——“体系结构”问题
- 如何实现乘法指令？...——“计算机组成”问题

- **计算机体系结构**
  - 程序员所见到的计算机系统的属性
  - 概念性的结构与功能特性
  - 比如指令系统、数据类型、寻址技术、I/O机理等

通用计算机系统的层次结构

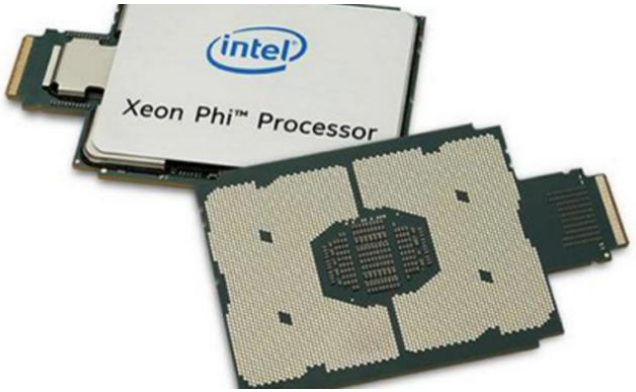


胡伟武等，《计算机体系结构基础》，2023-09

面向特定领域的体系结构（DSA）

- 通过根据领域特征调整体系结构，实现更高的效率
  - 不同于严格的ASIC，不需要对于每一个应用程序进行调整，而是对一个应用领域进行调整
  - 需要比通用处理器(general purpose)更多的特定领域知识(domain-specific knowledge)
- 示例：
  - 用于机器学习的神经网络处理器(Neural network processors for machine learning)

主流人工智能芯片

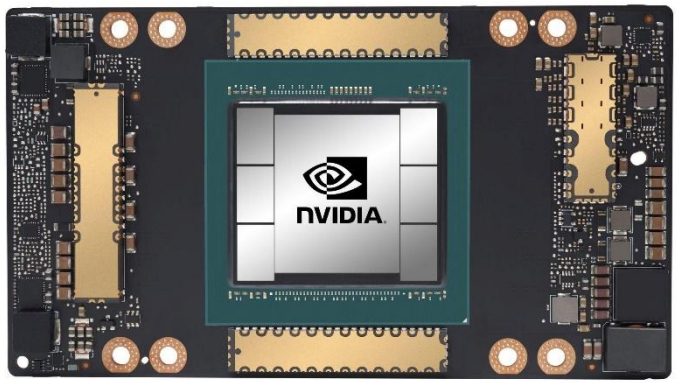


CPU: Central Processing Unit

Intel Xeon Phi processors

2013

Knights Landing (KNL) architecture

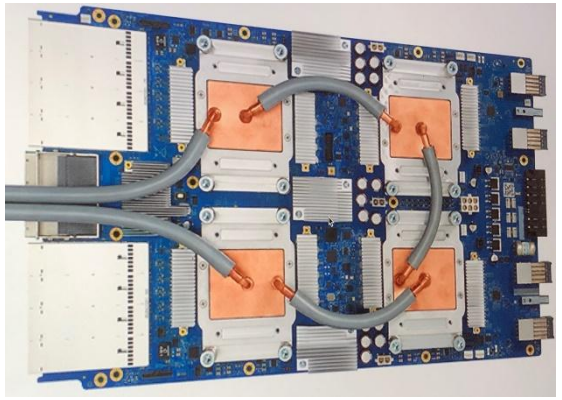


GPU: Graphics Processing Unit

NVIDIA GeForce 8800 GPU

2006

Along with CUDA



TPU: Tensor Processing Unit

Google's Tensor Processing Unit (TPU)

2013

TPUv1 or TPU Accelerator

以及其他ASIC加速芯片，如寒武纪（中科院）、昇腾（华为）、IPU（Graphcore）、FPGA相关加速器

主流人工智能芯片



2015-02-01



2018-01-01





2022-01-01



2025-09-27

人工智能芯片对比

	Training		Inference		Generality[88]	Inference accuracy[89]
	Efficiency	Speed	Efficiency	Speed		
CPU	1x baseline				Very High	~98-99.7%
GPU	~10-100x	~10-1,000x	~1-10x	~1-100x	High	~98-99.7%
FPGA	-	-	~10-100x	~10-100x	Medium	~95-99%
ASIC	~100-1,000x	~10-1,000x	~100-1,000x	~10-1,000x	Low	~90-98%

Saif M. Khan, AI Chips: What They Are and Why They Matter, 2020

GPU（Graphics Processing Unit）

- **并行处理能力强**

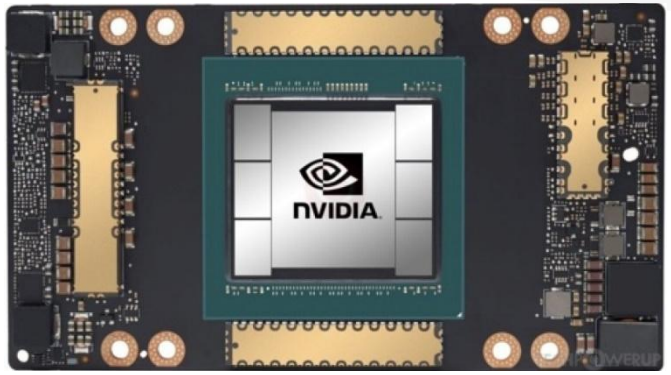
GPU拥有成百上千个核心，可以同时执行大量的计算任务，特别适合于处理图形渲染等需要高度并行处理的工作。

- **专门化**

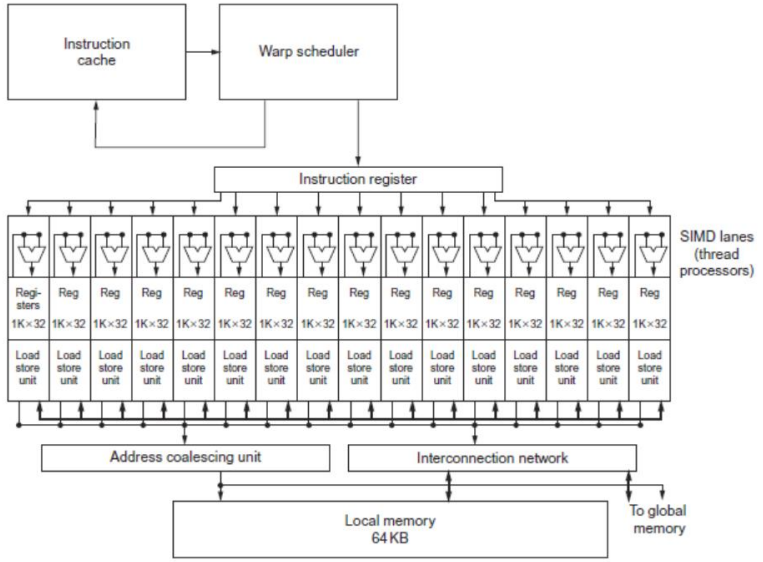
GPU最初设计用于加速图形渲染，但现在也广泛应用于通用计算（GPGPU），如机器学习、科学模拟等。

- **高吞吐量**

GPU设计注重于数据吞吐量，适合处理大量数据运算。



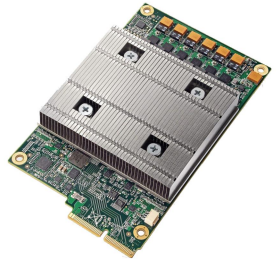
NVIDIA A100 SXM 80GB GPU

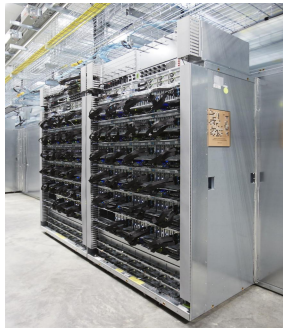


GPU架构设计

Tensor Processing Unit（TPU）

张量处理单元（英文：Tensor Processing Unit，简称：TPU），是Google 开发的专用集成电路（ASIC），专门用于加速机器学习。TPU已被大规模应用于搜索查询、机器翻译，也曾被用于AlphaGo的围棋比赛中。





- The Matrix Unit: 65,536 (256x256) 8-bit multiply-accumulate units
- 700 MHz clock rate
- Peak: 92T operations/second
- 65,536 * 2 * 700M
- >25X as many MACs vs GPU
- >100X as many MACs vs CPU
- 4 MiB of on-chip Accumulator memory
- 24 MiB of on-chip Unified Buffer (activation memory)
- 3.5X as much on-chip memory vs GPU
- Two 2133MHz DDR3 DRAM channels
- 8 GiB of off-chip weight DRAM memory

65536个8位乘累加运算单元（MAC，乘积累加单元）

峰值：每秒92T(tera,万亿)次操作

拥有的MAC数量是GPU的25倍

拥有的MAC数量是CPU的100倍

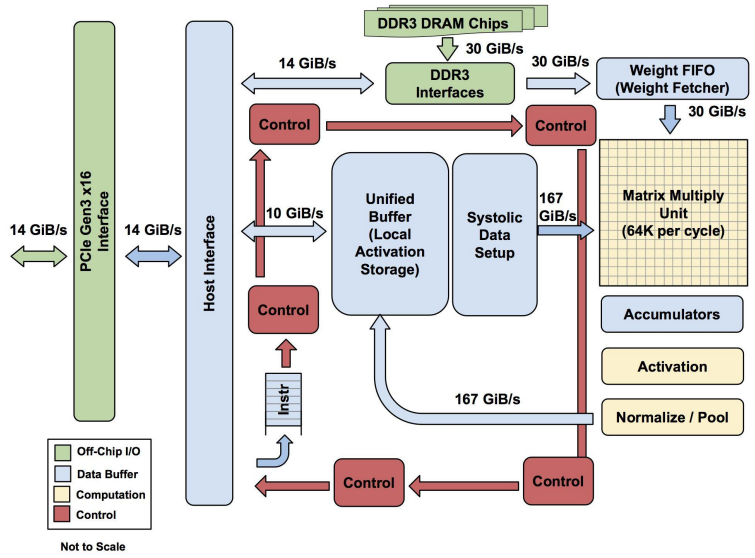
4 MiB的片上累加器内存

24 MiB的片上统一缓冲区

相对于GPU，片上内存容量是GPU的3.5倍

两个2133MHz的DDR3 DRAM通道

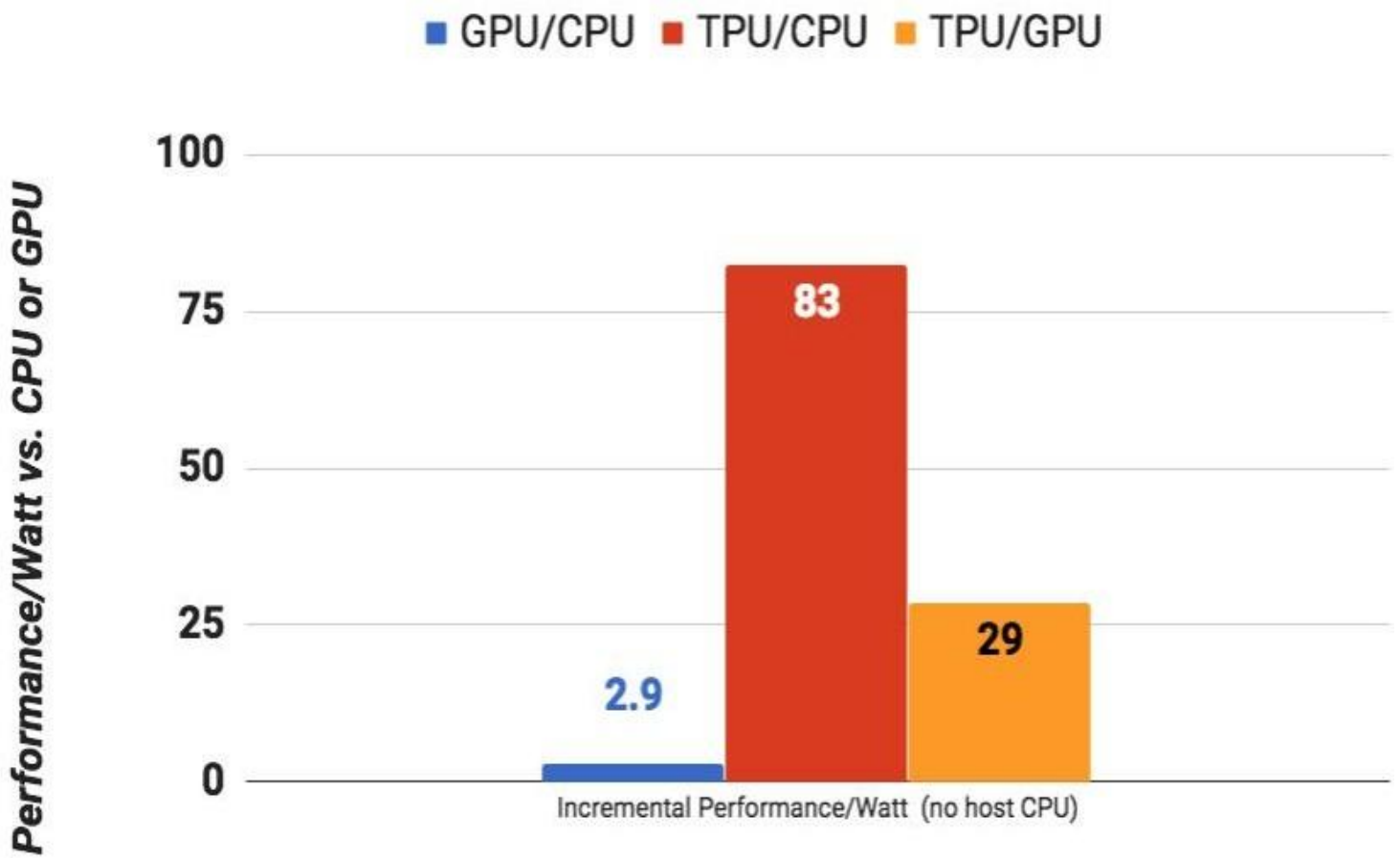
8 GiB的片外权重DRAM内存



TPU架构设计

Jouppi N P, Young C, Patil N, et al. In-datacenter performance analysis of a tensor processing unit[C]//Proceedings of the 44th annual international symposium on computer architecture. 2017: 1-12.

Perf/Watt TPU vs CPU & GPU



课程计划 – 2025年秋季 – 理论部分

章节	内容	学时	备注
第一章，绪论	绪论	1	
第二章，现代处理器体系结构	存储器层次结构设计	3	
	指令集并行与动态调度	3	
	分支预测、前瞻执行与静态调度	3	
第三章，AI处理器体系结构	向量处理机	2	
	GPU和CUDA编程	6	
	TPU、NPU和大规模集群	2	

课程考核

- **学时分配**
  - 理论课：20学时
  - 实验课：28学时
- **分数构成**
  - 平时测验：20
  - 实验：40
  - 期末考试：40

考核环节	所占分值	考核与评价细则
(1) 平时测验	20	1）平时随堂抽查提问、测试等。
(2) 实验	40	1）按要求完成3个实验项目 2）接受现场验收(演示与提问回答) 3）最后一次实验课上验收，基本用例通过及提问；上交实验报告，最终一份报告呈现三次实验过程及结果 评价标准：代码是否规范，功能是否完善，报告是否完整，图表是否清晰等。
(3) 期末考试(闭卷)	40	期末考试： 1）计算机体系结构概念原理的理解 2）核心算法的掌握 3）按学时比例分配各章节的考核知识点与分值。
课程最终成绩 = （1）+（2）+（3）		



目录 | CONTENT

1 课程介绍

2 计算机体系结构的发展历史

3 体系结构面临的挑战和发展机遇



Patterson and Hennessy win 2017 Turing!

A New Golden Age for Computer Architecture

By John L. Hennessy, David A. Patterson, Communications of the ACM, February 2019, Vol. 62 No. 2, Pages 48-60

1960年早期IBM遇到的兼容性问题(Compatibility Problem)

- 20世纪60年代初，仅IBM就存在4个不兼容的计算机系列！

  701 → 7094

  650 → 7074

  702 → 7080

  1401 → 7010

每个系列都有自己独立的：

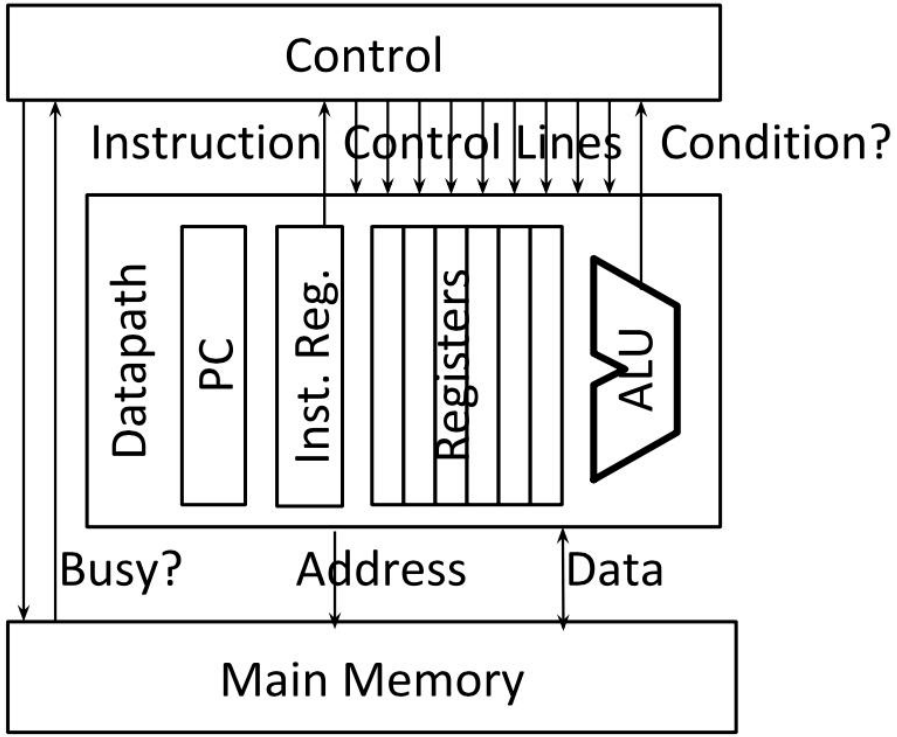
- 指令集ISA (Instruction set architecture)
- I/O 系统和外部存储（Secondary Storage ）：磁带（ magnetic tapes ），磁鼓（ drums ）和磁盘 (disks)
- 汇编器（Assemblers），编译器（compilers），开发库 (libraries) 等
- 利基市场（Market niche）：商业应用（ business ），科学计算（ scientific ），实时系统 (real time) 等



IBM System/360 – 统一的指令集架构 (one ISA to rule them all)

控制器(Control) VS 数据通路(Datapath)

- 处理器设计将**数据通路**(Datapath)和**控制器**(Control)作为两个独立单元分开设计。其中数据通路是存储数据和进行算术运算的地方，而控制器则负责对数据通路中的操作进行排序（ sequences operations on datapath）
- 因此，计算机设计者面临的最大挑战：如何正确地设计控制器



毛里斯·威尔克斯 (Maurice Wilkes) 发明了**微程序(microprogramming)** 的概念，用于设计处理器的控制器*。

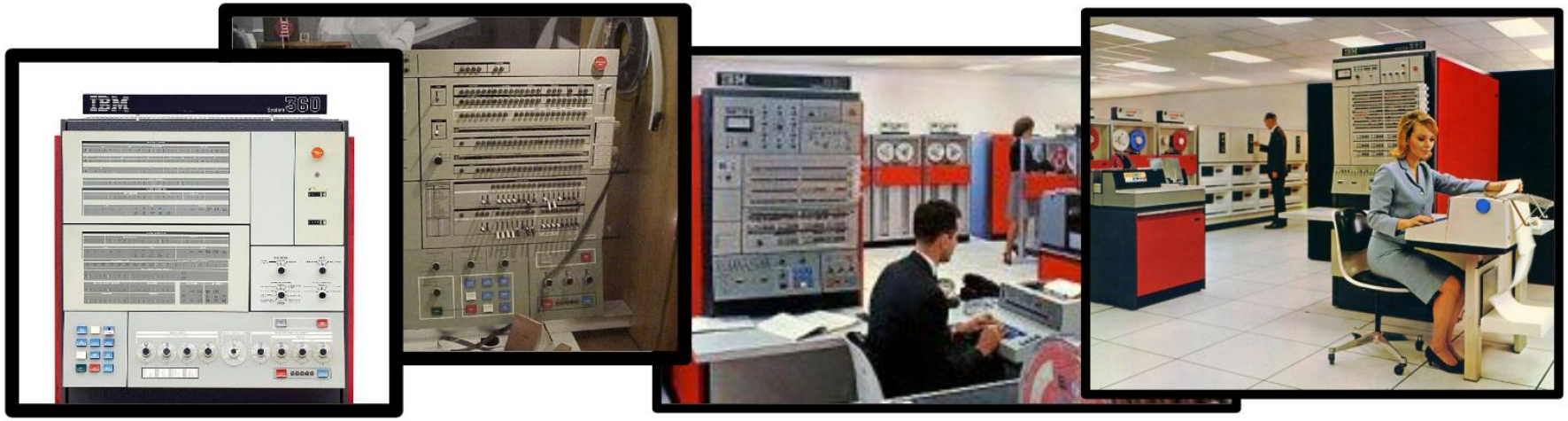


- 逻辑元件昂贵 VS ROM或RAM
- 只读存储器(ROM)比随机存取存储器(RAM)便宜
- 只读存储器(ROM)比随机存取存储器(RAM)快得多

* “Micro-procramming and the design of the control circuits in an electronic digital computer." M. Wilkes, and J. Stringer. Mathematical Proc. of the Cambridge Philosophical Society, Vol, 49, 1953

IBM 360使用的微程序

Model	M30	M40	M50	M65
Datapath width	8 bits	16 bits	32 bits	64 bits
Microcode size	4k x 50	4k x 52	2.75k x 85	2.75k x 87
Clock cycle time (ROM)	750 ns	625 ns	500 ns	200 ns
Main memory cycle time	1500 ns	2500 ns	2000 ns	750 ns
Price (1964 $)	$192,000	$216,000	$460,000	$1,080,000
Price (2018 $)	$1,560,000	$1,760,000	$3,720,000	$8,720,000





Fred Brooks, Jr.

Fred Brooks

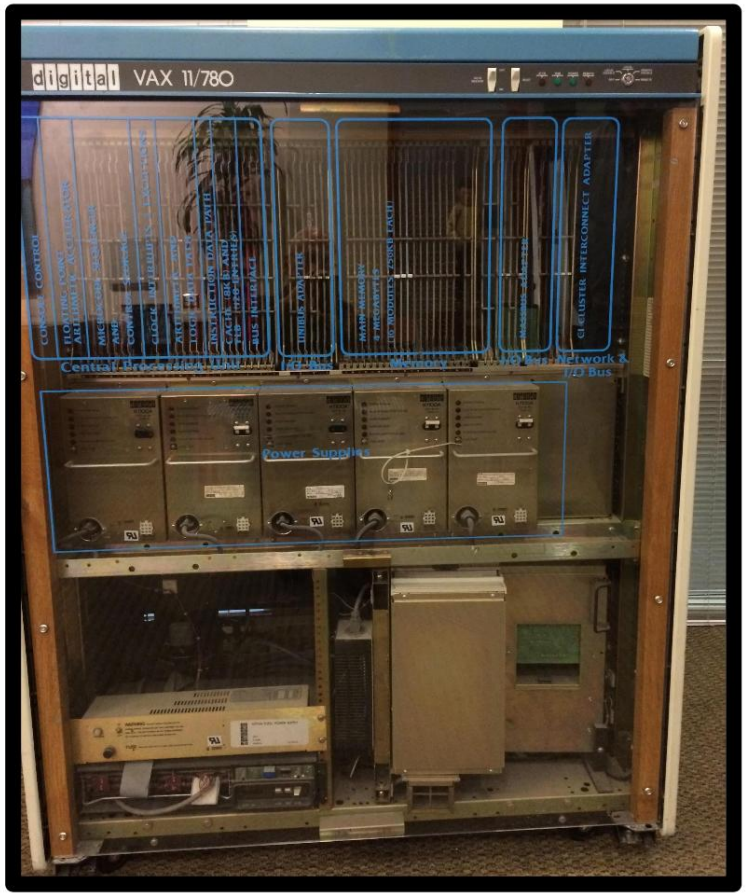
- 1953年，获得杜克大学物理学学士学位
- 1955年，获得哈佛大学计算机科学硕士学位
- 1956年，获得哈佛大学应用数学和计算机科学博士学位
- 1956年-1965年，任职于IBM公司
  - 参加了Harvest和STRETCH计算机的开发，他和同事提出了“可屏蔽中断”的概念，使用户击键时字符显示在屏幕上，而计算机的运行却不会混乱，并申请了专利，该系统被广泛采用为计算机的基本机制
  - 帮助设计了IBM 7090超级计算机
  - 帮助设计IBM 8000，这是晶体管大型计算机
  - 带领团队设计IBM System/360的产品线
- 1976年，当选为美国国家工程院院士，同年当选为美国艺术与科学院院士
- 1999年，获得图灵奖





集成电路技术(IC Technology)、微指令 (Microcode)和复杂指令集(CISC)

- 逻辑元件(Logic)、随机存取存储器(RAM)、只读存储器(ROM)都是使用相同的晶体管(transistors)实现的
- 半导体随机存取存储器(Semiconductor RAM)的速度与只读存储器(ROM)几乎相同
- 随着摩尔定律(Moore's Law)的发展，用于控制指令存储的内存容量可以增长
- 由于使用随机存取存储器(RAM)，更容易修复微指令(Microcode)错误
- 允许更复杂的指令集架构，即复杂指令集 (CISC)
- 小型计算机 (即TTL服务器) 示例：
  - 数字设备公司 (Digital Equipment Corp)(DEC)
  - 在1977年推出的VAX ISA
- 具有5K x 96b 的微指令



可编写控制存储 (Writable Control Store)

- 如果控制存储是随机存取存储器(RAM)，那么可以根据应用定制“固件” (firmware)：即“可编写控制存储” (Writable Control Store)
- 微编程(Microprogramming)在学术界变得流行
  - 帕特森博士论文 (Patterson PhD thesis)[1]
  - SIGMICRO是为微编程而专设的[2]
- 1973年的施乐阿尔托 (Xerox Alto) (使用位切片TTL逻辑门电路技术)
  - 第一台具有图形用户界面(Graphical User Interface, GUI)与以太网(Ethernet)的计算机
  - 位块传送 (BitBlt) 和以太网控制器(Ethernet controller)在微指令中实现





Chuck Thacker

[1] Verification of microprograms, David Patterson, UCLA, 1976
[2] The design of a system for the synthesis of correct microprograms, David Patterson, Proc. 8th Annual Workshop of Microprogramming, 1975

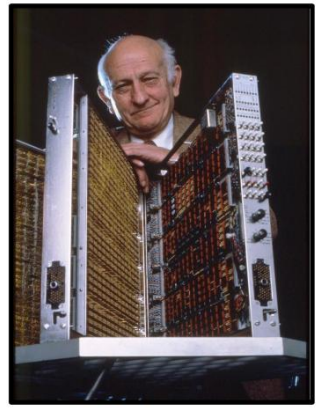
微处理器(Microprocessor)的演变

- 1970年代快速发展，得益于金属氧化物半导体 (MOS) 技术的进步，模仿小型计算机和大型机的指令集架构(ISAs)
- “微处理器战争”：通过添加指令（微代码易于实现）进行竞争，这在汇编语言编程中是合理的
- Intel iAPX 432：1970年代最具雄心的微处理器，始于1975年
  - 32位基于容量的面向对象架构(32-bit capability-based object-oriented architecture)，使用Ada编写的定制操作系统
  - 性能不佳，复杂性（多芯片）和可用性问题；公布于1981年
- Intel 8086（或叫iAPX 86, 1978年，8MHz，29,000个晶体管）
  - “权宜之计”的16位处理器，52周形成的新芯片
  - 指令集架构在3周内（10人周）设计完成，与8位8080汇编兼容
- 1981年IBM个人计算机选择了Intel 8088 进行8位总线（之后是摩托罗拉 68000）
- 预估PC个人计算机销售量：250,000
- 实际PC个人计算机销售量：100,000,000 →8086“一夜”成功
- 个人计算机软件的二进制兼容性 → 为8086的未来带来了光明前景



分析1980年代的微指令机器(Micro-coded Machines)

- 世界从汇编编程转向了高级语言编程(HLL programming)
  - 编译器(Compilers)现在成为了衡量程序性能的主要来源
- IBM的John Cocke小组
  - 他们在IBM内部研究简单的流水线处理器，801小型计算机（ECL服务器），以及高级编译器
  - 他们将他们的编译器移植到IBM 370，只使用简单的寄存器-寄存器(register-register)和加载/存储指令(load/store)（类似于801）
  - 比使用完整的370指令集的现有编译器快3倍!
- 1980年代初DEC公司的Emer和Clark[1]
  - 他们发现VAX 11/180的平均时钟周期每指令 (CPI) = 10 ！
  - 他们发现VAX指令集的20% = 微指令的60%，但只占执行时间的0.2% ！
- 1979年DEC的Patterson在微处理器中修复微指令错误？[2]
  - 可编写控制存储(Writable Control Store)中的ISA解释器有什么神奇之处？为什么不是其他程序？

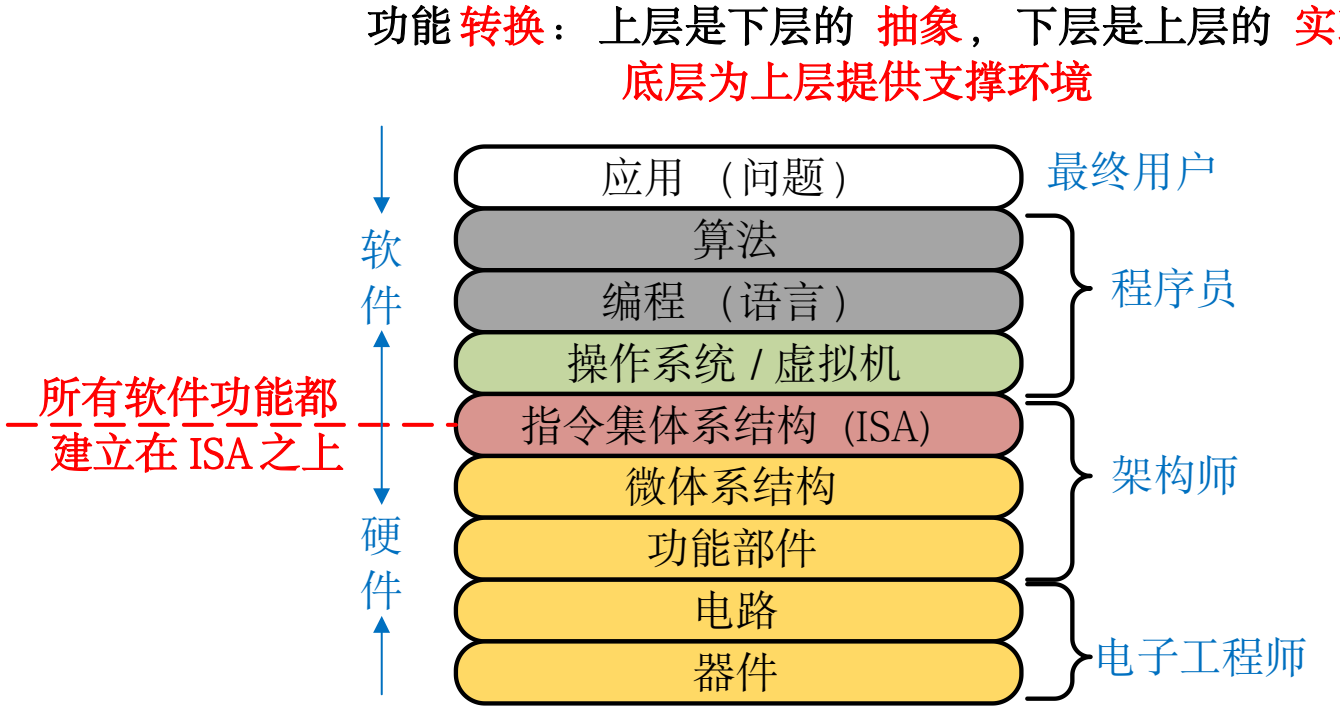


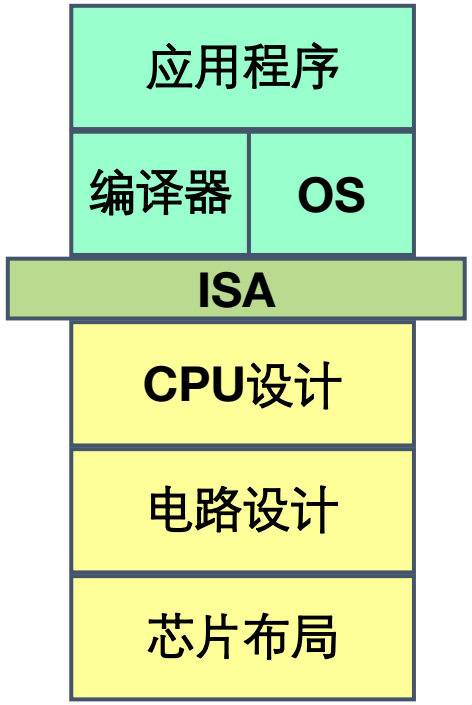
John Cocke

[1] "A Characterization of Processor Performance in the VAX-11/780." J. Emer and D.Clark, ISCA, 1984
[2] "RISCy History" David Patterson, May 30, 2018, Computer Architecture Today Blog

指令集体系结构/架构（ISA）

- ISA定义了一台计算机可以执行的所有指令的集合，每条指令规定了计算机执行什么操作，所处理的操作数存放的地址空间以及操作数类型。
- ISA是计算机体系结构中与程序设计有关的部分，包含基本数据类型，指令集，寄存器，寻址模式，存储体系，中断，异常处理以及外部IO。
- 指令集架构与微架构（microarchitecture）不同，微架构指的是一套用于执行指令集的微处理器设计方法，使得指令集架构可以在微处理器上被执行。





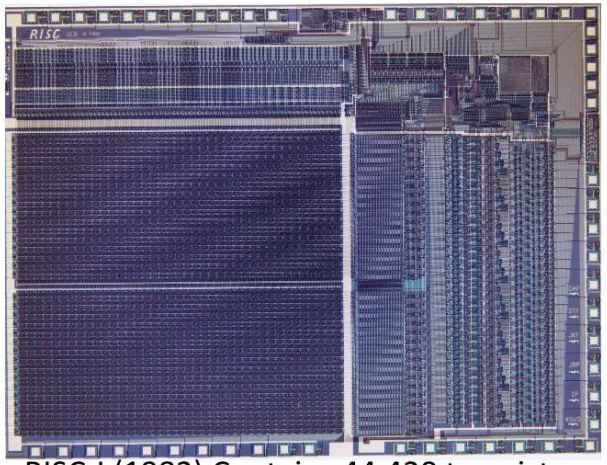
从CISC(Complex Instruction Set Computer)到RISC(Reduced …)



- 使用SRAM作为用户可见指令的指令缓存(instruction cache)
  - 快速指令内存的内容会根据应用程序现在的需要而改变，而不是指令集架构解释器(ISA interpreter)
- 使用简单的指令集架构 (Use simple ISA)
  - 指令和微指令一样简单。
  - 编译代码只使用了一些CISC指令。
  - 启用流水线实现(Enable pipelined implementations)
- 芯片集成的进一步优势
  - 在80年代初，终于可以将32位数据通路(32-bit datapath)+小缓存(caches)放在一个单独的芯片上
- Chaitin的寄存器分配方案[1]对加载-存储指令集架构有利

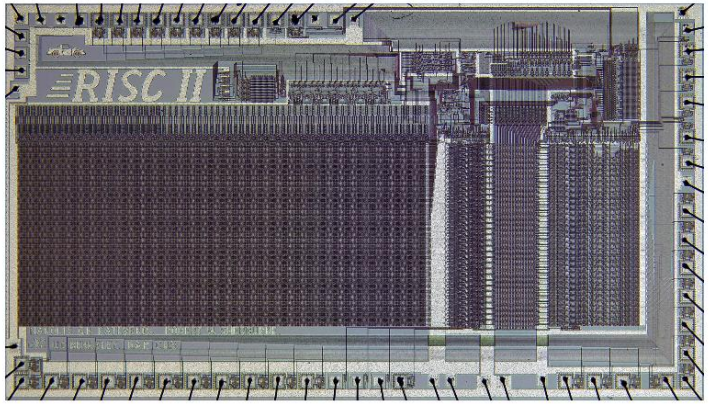
[1] Chaitin, Gregory J., et al. "Register allocation via coloring." Computer languages 6.1 (1981), 47-57.

Berkeley & Stanford 的 RISC 芯片

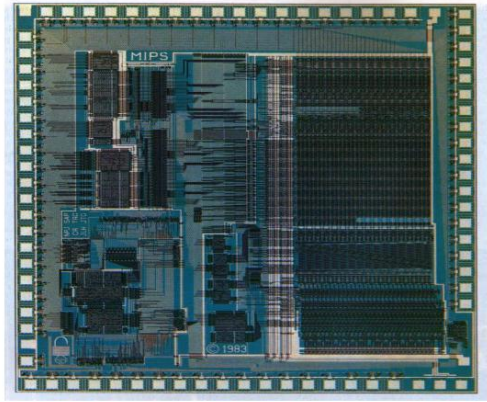




RISC-I (1982) Contains 44,420 transistors, fabbed in 5 µm NMOS, with a die area of 77 mm^2, ran at 1 MHz



RISC-II (1983) contains 40,760 transistors, was fabbed in 3 µm NMOS, ran at 3 MHz, and the size is 60 mm^2





Stanford MIPS (1983) contains 25,000 transistors, was fabbed in 3 µm & 4 µm NMOS, ran at 4 MHz (3 µm), and size is 50 mm^2 (4 µm)
(Microprocessor without Interlocked Pipeline Stages)

Fitzpatrick, Daniel, John Foderaro, Manolis Katevenis, Howard Landman, David Patterson, James Peek, Zvi Peshkess, Carlo Séquin, Robert Sherburne, and Korbin Van Dyke. "A RISCy approach to VLSI." *ACM SIGARCH Computer Architecture News* 10, no. 1 (1982):

Hennessy, John, Norman Jouppi, Steven Przybylski, Christopher Rowen, Thomas Gross, Forest Baskett, and John Gill. "MIPS: A microprocessor architecture." In *ACM SIGMICRO Newsletter*, vol. 13, no. 4, (1982).

David Patterson



- 1969年，获得加州大学洛杉矶分校数学专业学士学位
- 1970年，获得加州大学洛杉矶分校计算机科学专业硕士学位
- 1976年，获得加州大学洛杉矶分校计算机科学专业博士学位
- 1980 年代末，Patterson从DEC回到Berkeley，尝试发表RISC微处理器相关的论文，但论文被拒
  - Reviewer：这是一种愚蠢的设计计算机的方法，用这种方法设计计算机毫无意义，要额外的存储，要增加成本，还有打补丁，简直就是胡说八道
  - 从此Patterson决定启动RISC项目
- 1993年，当选为美国国家工程院院士
- 2017年，获得图灵奖

From: David PATTERSON [mailto:pattrsn@cs.berkeley.edu]
Sent: 05 January 2017 22:05
To: csshshi@Comp.HKBU.Edu.HK; qiangwang@Comp.HKBU.Edu.HK; pengfeixu@Comp.HKBU.Edu.HK; chxw@Comp.HKBU.Edu.HK
Subject: 4 questions about your paper

I was reading your paper "Benchmarking State-of-the-Art Deep Learning Software Tools" , https://arxiv.org/pdf/1608.07249.pdf and had some questions.

It's a very interesting paper. Congratulations. It must have been a lot of work. Are you submitting the paper to a conference or a journal?

Here are my technical questions:

1. Table 3 says 16 cores for Xeon and Table 4 shows 32 threads
but the text says "one 8-core server".
I presume the system is a 2-socket system, with two Xeon chips,
which leads to 16 cores and 32 threads. Is this correct?
Ans: Yes.

Dear Prof. Patterson,

It's really a surprise for me to receive the email from you.



David PATTERSON <pattrsn@cs.berkeley.edu>
发送至 Shaohuai、 Chu、 qiangwang、 pengfeixu@Comp.HKBU.Edu.HK

此邮件似乎是用英语撰写的
翻译成中文

I see that TensorFlow has just made a major release to version 1.0.
https://github.com/tensorflow/tensorflow/releases

Are you planning on using it in the next iteration of your study?
(Or have you done it already?)

Dave

处理器性能“铁律”：RISC是如何取胜的？

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Clock cycle}}$$

- CISC每个程序执行的指令数量更少（大约指令的3/4倍），但每条指令需要更多的时钟周期（大约6倍CPI）。

→ RISC约比CISC快4倍

Performance from architecture: comparing a RISC and a CISC with similar hardware organization, Dileep Bhandarkar and Douglas Clark, Proc. Symposium, ASPLOS, 1991.

Video of RISC History



Full ACM video is at http://bit.ly/2KKltJ5

CISC vs. RISC Today

PC时代

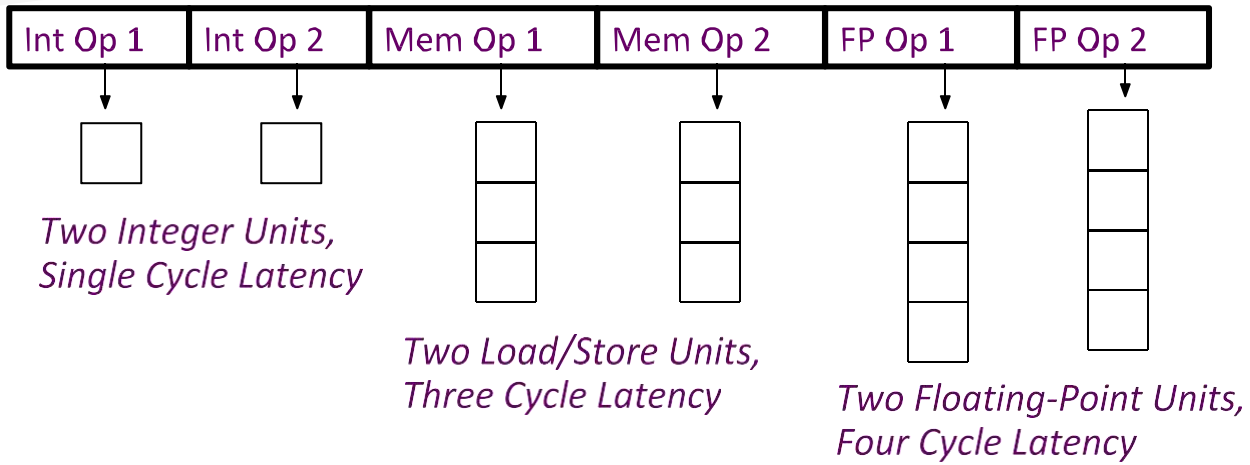
- 硬件将x86指令转化为内部的RISC指令
- 然后在MPU (Microprocessor Unit，微处理单元) 内部使用RISC技术
- > 350M / 年
- x86 ISA (指令集架构)最终在服务器和台式机上占据主导地位

后PC时代: Client/Cloud

- SoC (System-on-Chip，片上系统) vs. MPU
- 芯片的面积(area)，能耗(energy)和性能(performance)同样重要
- > 20B total / year in 2017
  - x86 PC的数量在2011年到达顶峰， 现在每年下降约8% (2016 < 2007)
  - x86 云服务器大约有10M台 (占云服务器总数20B的0.05%)
- 现在99%的处理器是RISC

*"A Decade of Mobile Computing", Vijay Reddi, 7/21/17, *Computer Architecture Today*

VLIW: Very Long Instruction Word, 超长指令字（Josh Fisher）



- Multiple operations packed into one instruction (like a wide microinstruction) 将多个操作打包成一条指令（类似于宽微指令）
- Each operation slot is for a fixed function 每个操作槽用于一个固定的功能
- Constant operation latencies are specified 指定了固定的操作延迟
- Architecture requires guarantee of: 架构要求保证
  - Parallelism within an instruction ⇒ no cross-operation RAW check 指令内部的并行性 ⇒ 没有跨操作的RAW检查
  - No data use before data ready ⇒ no data interlocks 数据在就绪前不允许使用 ⇒ 没有数据互锁

From RISC to Intel/HP Itanium, EPIC IA-64



- EPIC是Intel对其VLIW架构的命名
  - “Explicitly Parallel Instruction Computing” “明确并行指令计算”
  - A binary object-code-compatible VLIW 一个二进制目标代码兼容的VLIW
  - 从1994年开始与惠普(HP)共同开发
- IA-64是Intel 32位 x86 ISA的升级版
  - IA-64 = Intel Architecture 64-bit
  - AMD无法制造IA-64, 因此只能生产64位 x86
- First chip late (2001 vs 1997), but eventually delivered (2002) 首款芯片推迟到2002年最终交付
- 许多公司放弃了RISC，转向Itanium，因为人们普遍认为这是不可避免的 (Microsoft, SGI, Hitachi, Bull, …)

VLIW Issues and an “EPIC Failure”

- Compiler couldn’t handle complex dependencies in integer code (pointers) 编译器无法处理整数代码（指针）中的复杂依赖关系
- Code size explosion 代码量膨胀
- Unpredictable branches 无法预测的分支
- Variable memory latency (unpredictable cache misses) 变化的内存延迟（无法预测的缓存未命中）
  - Out of Order techniques dealt with cache latencies 乱序技术处理了缓存延迟
- Out of Order subsumed VLIW benefits 乱序技术取代了VLIW的优势
- “The Itanium approach...was supposed to be so terrific until it turned out that the wished-for compilers were basically impossible to write.” - Donald Knuth, Stanford
- Pundits noted delays and under performance of Itanium product ridiculed by the chip industry 专家们指出Itanium产品的延迟和性能不佳，被芯片行业嘲笑



Itanimum ⇒ “Itanic” (like infamous ship Titanic)

第一部分总结: Consensus on ISAs Today



- Not CISC: 近30年没有出现新的通用CISC ISA
- Not VLIW: 近15年没有推出新的通用VLIW ISA，VLIW在通用计算领域是失败的
  - Complex VLIW architectures close to in-order superscalar in complexity, no real advantage on large complex apps 复杂的VLIW架构与顺序超标量架构相近，对于大型复杂应用程序没有真正的优势
  - Although VLIWs successful in embedded DSP market (Simpler VLIWs, easier branches, no caches, smaller programs) 尽管VLIW在嵌入式DSP（Digital Signal Processing，数字信号处理器）市场上取得了成功（简单的VLIW架构、更容易的分支处理、无缓存、更小的程序）
- RISC！人们普遍认为RISC对于通用指令集架构是最好的选择！



目录 | CONTENT

1 课程介绍

2 计算机体系结构的发展历史

3 体系结构面临的挑战和发展机遇

Fundamental Changes in Technology

摩尔定律 (Moore's Law)：

摩尔定律得名于英特尔(Intel)的联合创始人戈登·摩尔。在1965年，摩尔提出了一个观察结果，即集成电路上可容纳的晶体管数量大约每两年翻一番。

阿姆达尔定律 (Amdahl's Law)

阿姆达尔定律由吉恩·阿姆达尔提出，对于一个并行化的系统，总体的性能提升受限于系统中不能并行化的那部分。阿姆达尔定律强调了优化程序中最慢的部分的重要性，并指出了扩展并行处理能力的潜在限制。

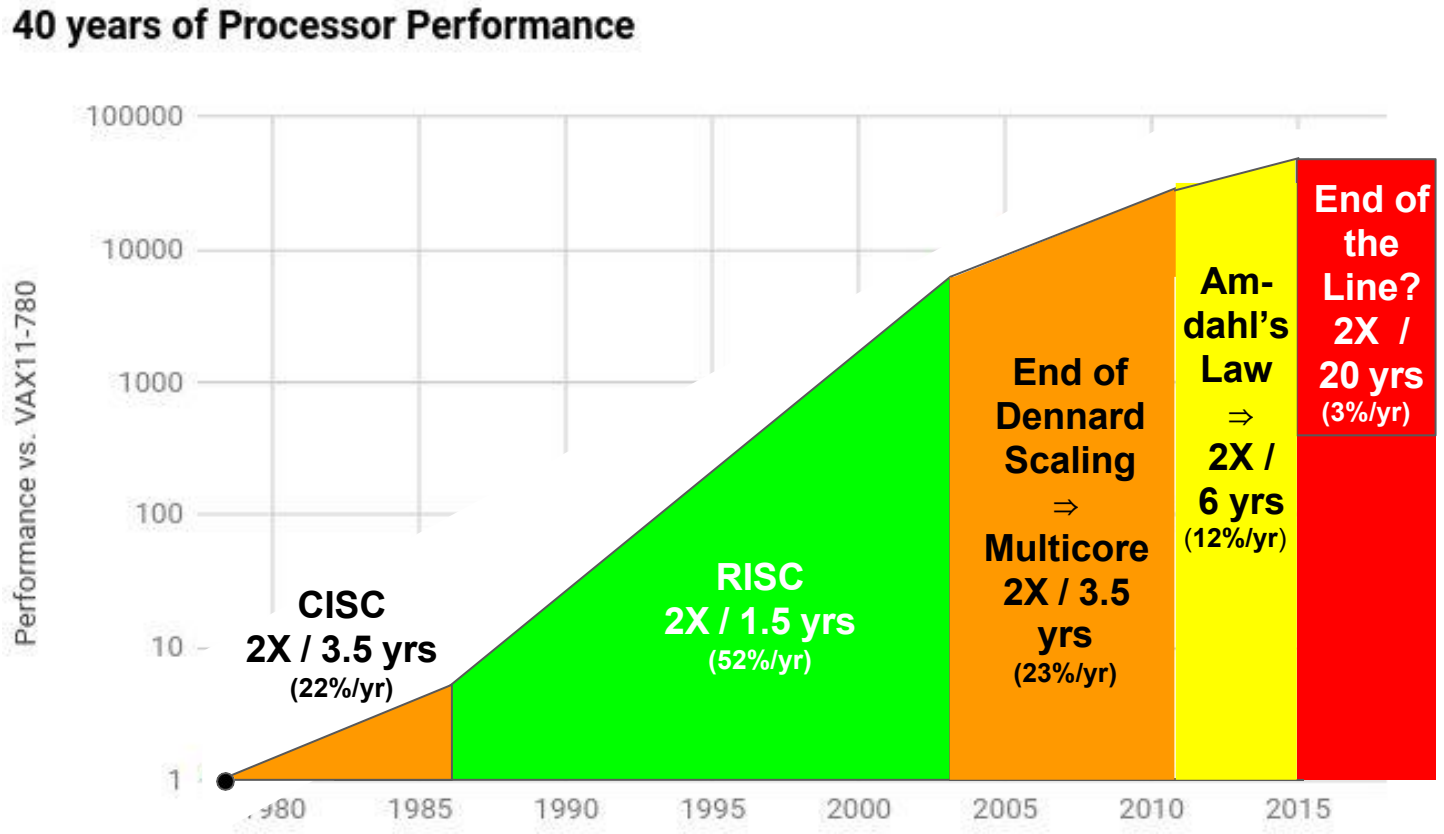
登纳德定律 (Dennard Scaling)

随着晶体管尺寸缩小，它们的功耗密度保持不变，更小的晶体管会消耗更少的能量，并且可以更快地运行。这使得制造商能够把更多的晶体管集成到同样大小的芯片上，而不增加功耗。

Fundamental Changes in Technology

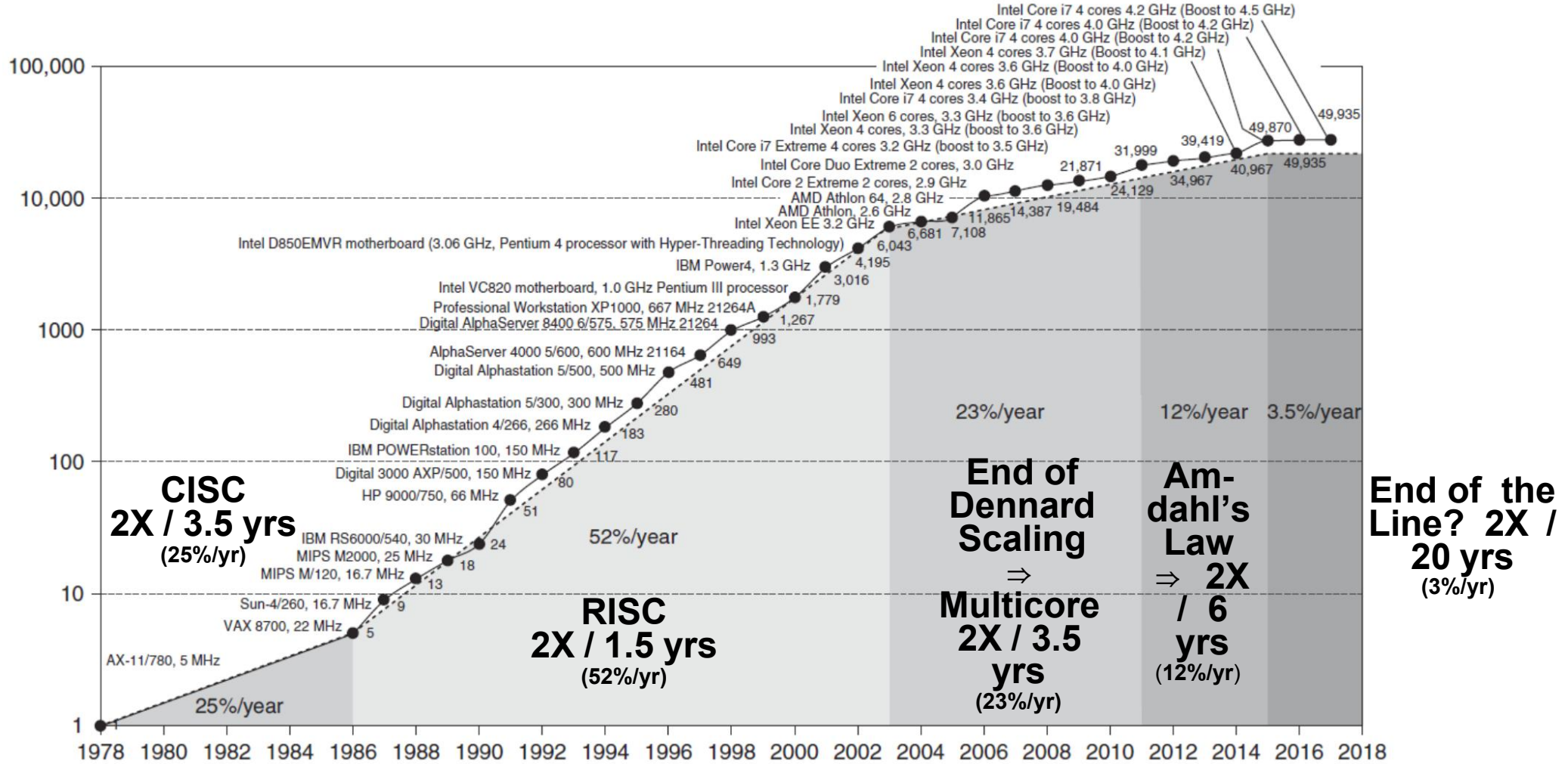
- Technology
  - End of Dennard scaling: power becomes the key constraint Dennard缩放定律的结束：功耗成为主要限制因素
  - Ending of Moore's Law: transistors improvement slows 摩尔定律的结束：晶体管改进放缓
- Architectural
  - Limitation and inefficiencies in exploiting instruction level parallelism end the uniprocessor era in 2004 利用指令级并行性的限制和低效性使得单处理器时代于2004年结束
  - Amdahl's Law and its implications end "easy" multicore era 阿姆达尔定律及其影响结束了"轻松"的多核时代
- Products
  - PC/Server ⇒ IoT, Mobile/Cloud 个人电脑/服务器 ⇒ 物联网、移动设备/云计算

End of Growth of Single Program Speed?

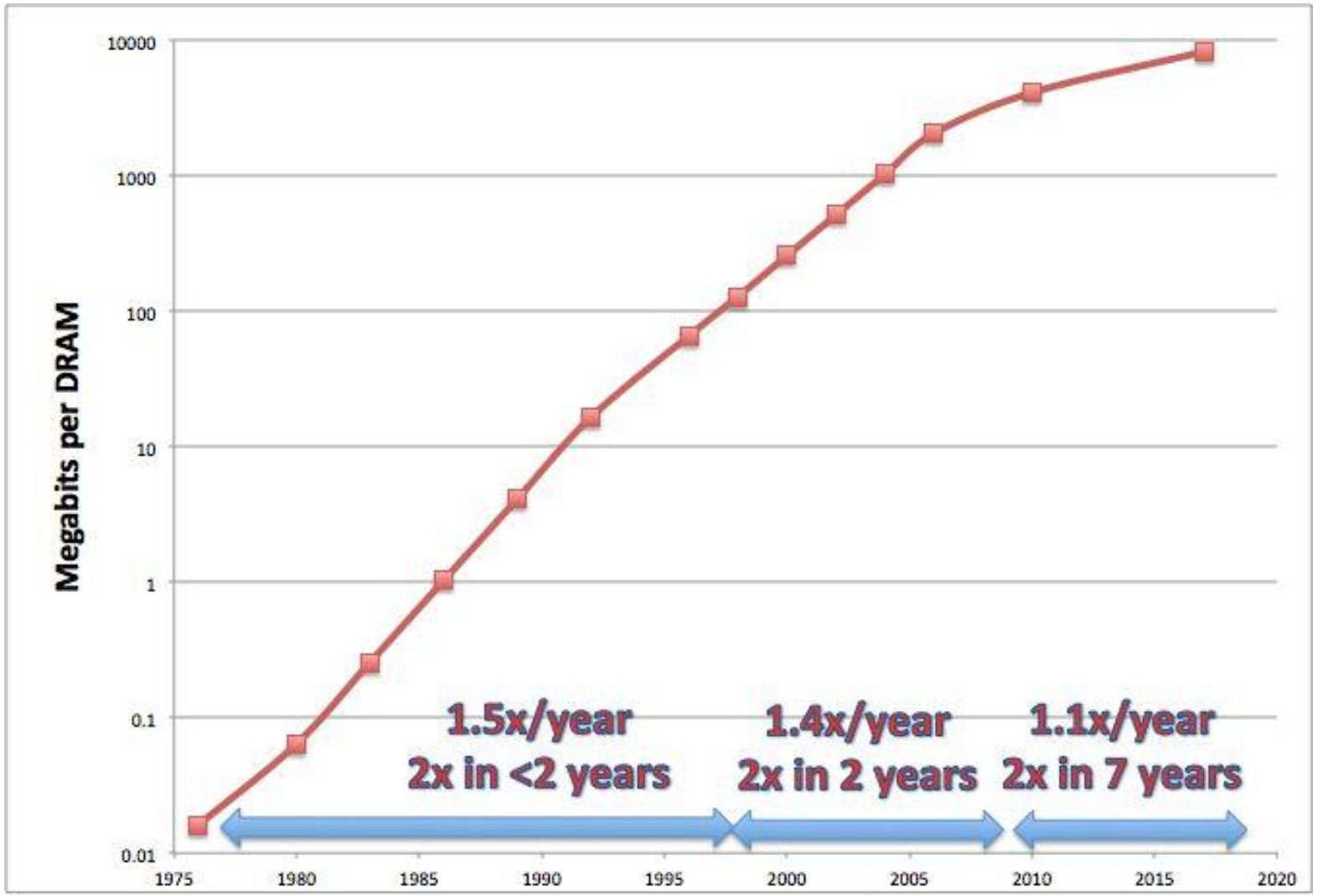


Based on SPECintCPU. Source: John Hennessy and David Patterson, Computer Architecture: A Quantitative Approach, 6/e. 2018

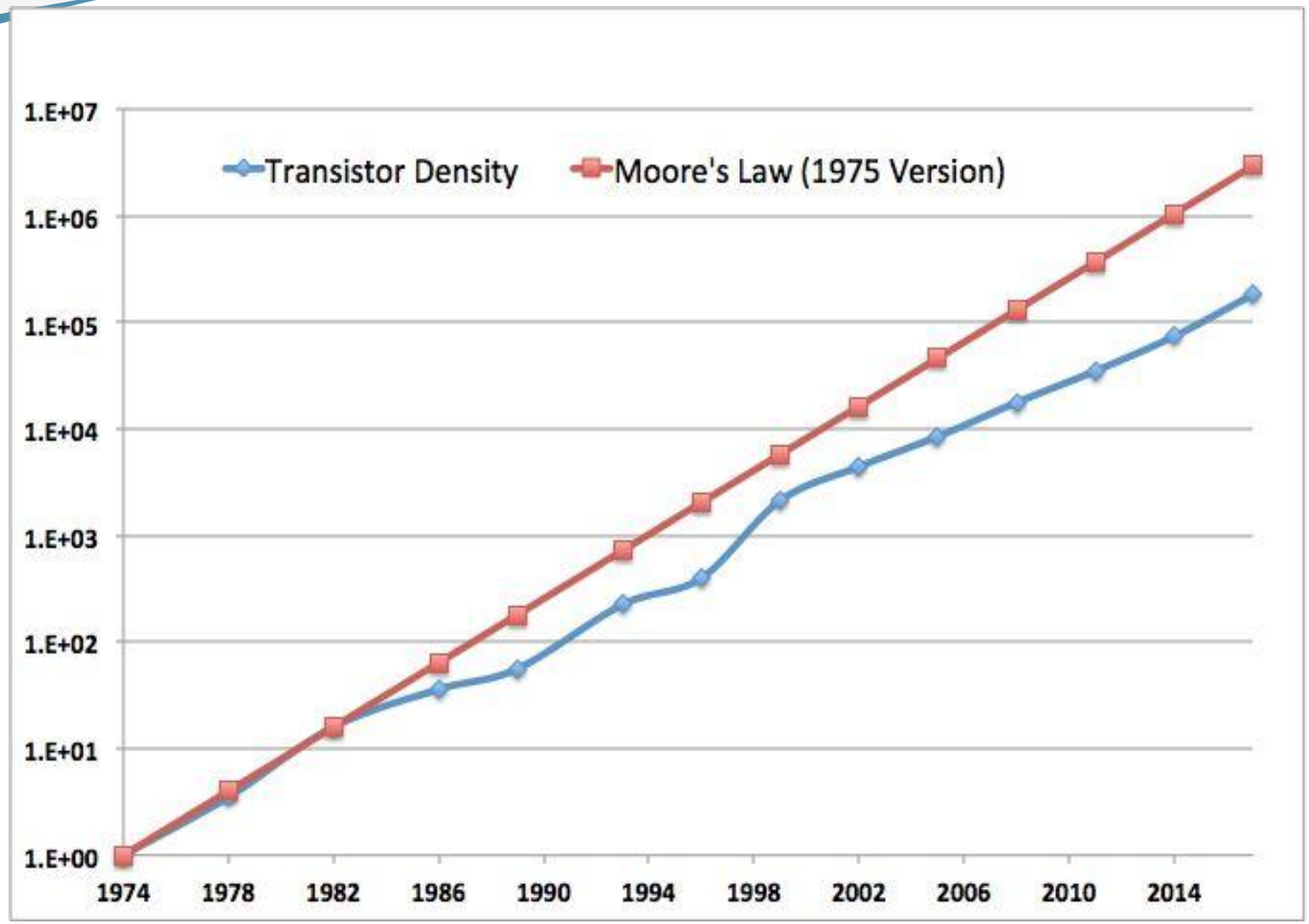
Moore's Law in Processors



Moore's Law in DRAMS



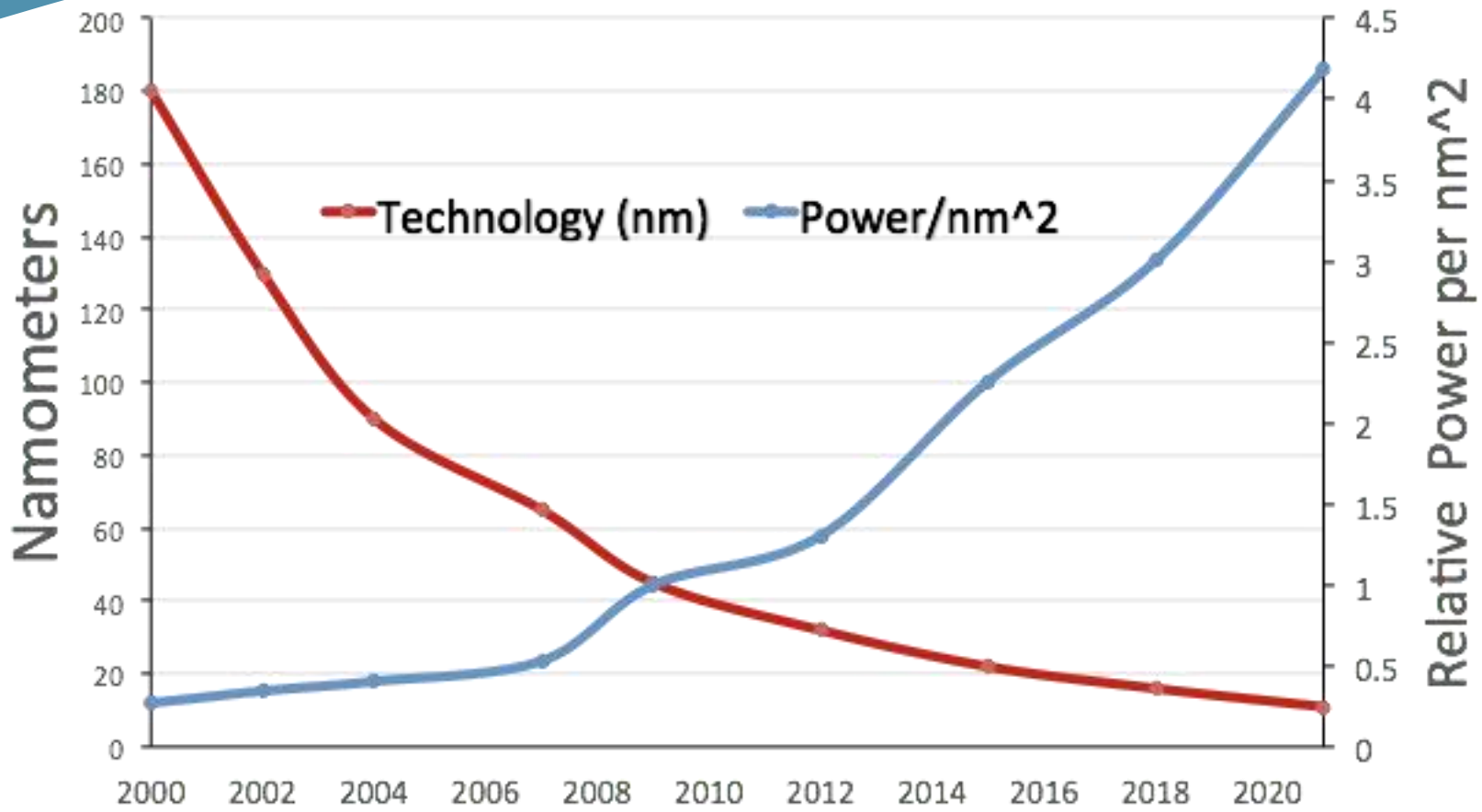
Moore’s Law Slowdown in Intel Processors



Cost/transistor slowing down faster, due to fab costs.

由于制造成本的上升，每个晶体管的成本降低的速度正在减慢，而且这种减慢的速度比预期的还要快。

Technology & Power: Dennard Scaling

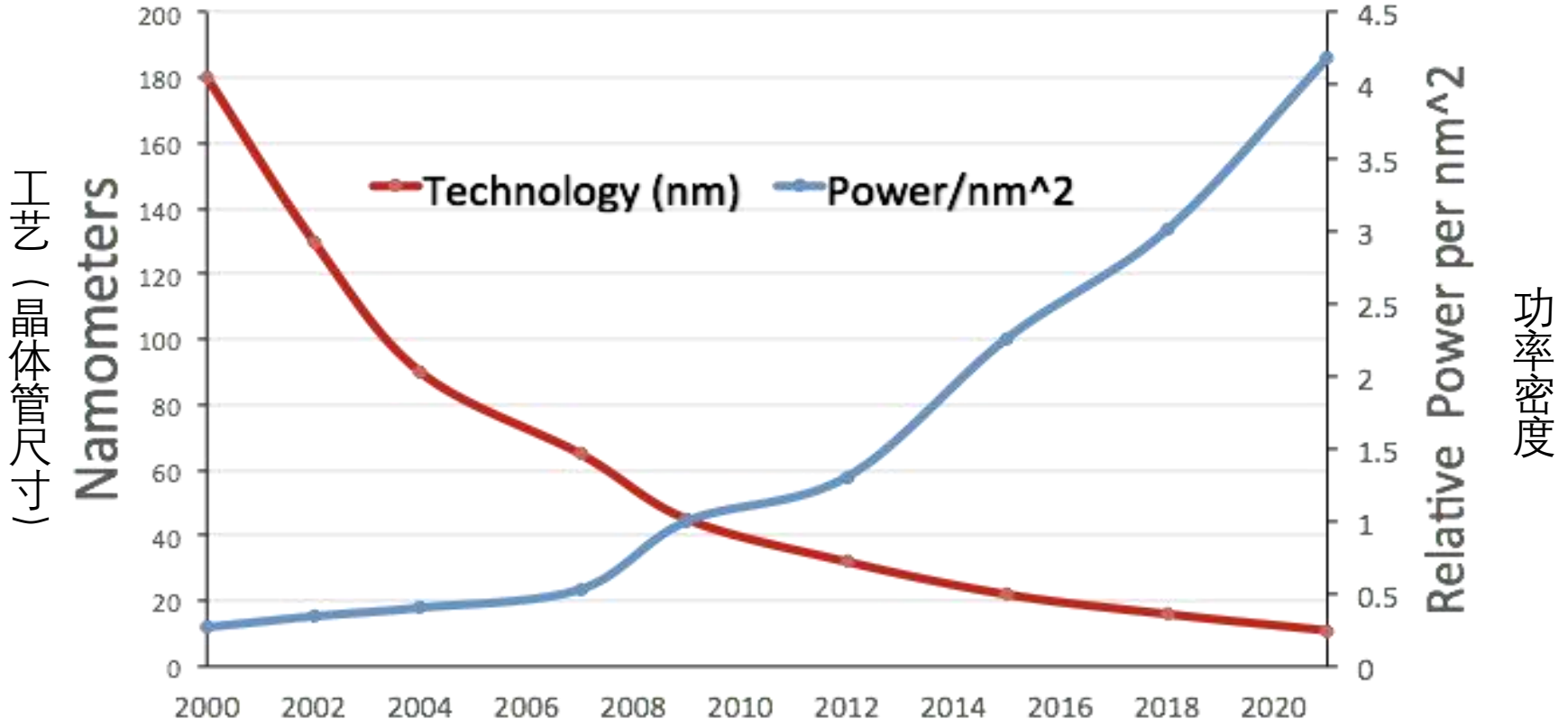


Energy scaling for fixed task is better, since more and faster transistors

对于固定任务而言，能源的扩展更好，因为有更多和更快的晶体管

Power consumption based on models in "Dark Silicon and the End of Multicore Scaling," Hadi Esmaelizadeh, *ISCA,* 2011

工艺与能耗：登纳德缩放定律(Dennard Scaling)



因为使用的晶体管更多且更快，在固定任务上的能量缩放表现更好

1.Power consumption based on models in "Dark Silicon and the End of Multicore Scaling," Hadi Esmaelizadeh, *ISCA,* 20111

现有挑战总结

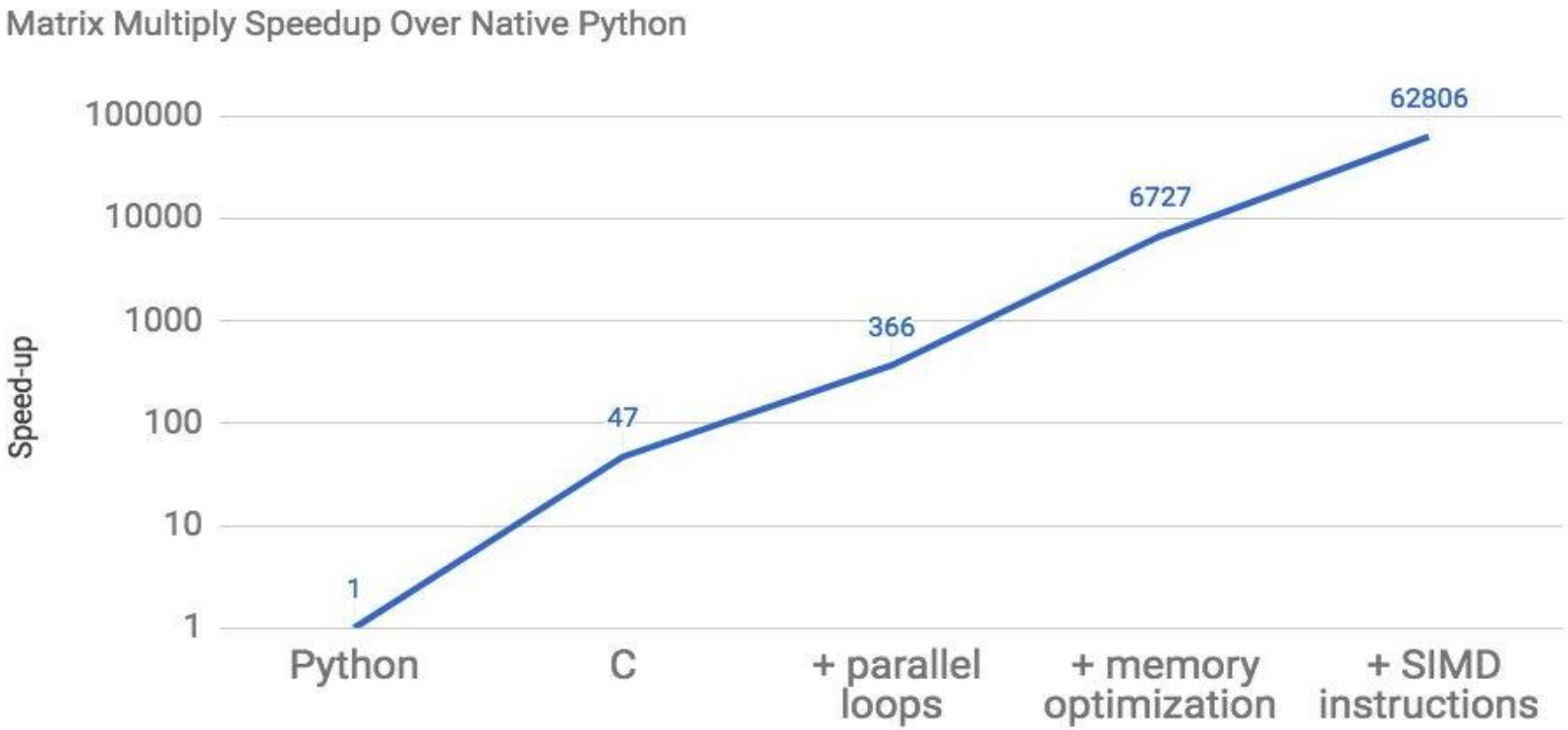
- 性能改进停滞不前
  - 摩尔定律减缓
  - 登纳德缩放定律不再适用
    - 微体系结构技术：ILP(Instruction Level Parallelism,指令集并行)、多核等，效率低下，因此消耗能量
- 计算机安全状况令所有计算机领域的人都感到尴尬
  - 似乎不太可能使用仅限软件的解决方案使系统变得安全

还有哪些机遇?

- 以软件为中心
  - 现代脚本语言是可解释的、动态类型化的，并鼓励重用
  - 对程序员有效，但对执行无效
- 以硬件为中心
  - 只剩下面向特定领域体系结构(*Domain Specific Architectures*)
  - 只做一些任务，但做得非常好
- 软硬件结合
  - 面向特定领域的语言和体系结构(Domain Specific Languages & Architectures)

机遇是什么?

矩阵乘法: 相对python版本的实现的加速 (18核Intel)



from: “There’s Plenty of Room at the Top,” Leiserson, et. al., *to appear*.

矩阵乘法(1)：python实现

我们来看一个简化的例子，这个例子将通过几个阶段来展示如何优化一个二维矩阵乘法运算。

假设我们有两个矩阵A（n*m）和B(m*p)，我们要计算它们的乘积C(n*p)。

```python
def matrix_multiply_python(A, B):
    n, m = len(A), len(A[0])
    m, p = len(B), len(B[0])
    C = [[0] * p for _ in range(n)]
    for i in range(n):
        for j in range(p):
            for k in range(m):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

我们要对其进行优化

矩阵乘法(2)：C语言(Use a Compiled Language)

下一步是用 C 语言重新编写上述算法。C 语言提供了更接近硬件的控制，以及更高的执行效率：

```c
void matrix_multiply_c(int n, int m, int p, int **A, int **B, int **C) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p; j++) {
            C[i][j] = 0;
            for (int k = 0; k < m; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

矩阵乘法(3)：并行计算

我们使用并行编程框架，如 OpenMP，可以允许多个核心同时工作。

下面的 C 代码示例使用了 OpenMP，#pragma omp parallel for 指令来并行化最外层的循环：

```c
#include <omp.h>

void matrix_multiply_parallel(int n, int m, int p, int **A, int **B, int **C) {
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p; j++) {
            C[i][j] = 0;
            for (int k = 0; k < m; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

矩阵乘法(4)：内存访问优化

在此阶段，我们可能需要重新组织数据结构或改变循环的顺序，以优化内存访问模式：

```
// 使用循环交换，优化内存访问模式
void matrix_multiply_memory_opt(int n, int m, int p, int **A, int **B, int **C) {

    // 这个循环与之前的顺序不同
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < m; k++) {
            for (int j = 0; j < p; j++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

在内存优化的版本中，我们通过调整循环的顺序来优化缓存的使用。这种循环重排被称为循环交换（loop interchange）。这是一种很常见的优化，它试图确保内部循环访问连续的内存位置，从而增加缓存命中率。

矩阵乘法(5)：SIMD 指令集

最后，我们可以利用 SIMD 指令集，如 Intel 的 AVX。在C/C++中，使用SIMD通常意味着利用这些扩展提供的内建函数或直接用汇编语言编程。下面是使用 C 语言和 SIMD 指令的示例：

```
#include <immintrin.h>

void matmul_simd(float* A, float* B, float* C, const int n, const int m, const int p) {
    // 假设所有矩阵都已正确对齐至32字节边界。
    // 假设n, m, 和 p 都是8的倍数，这样可以使用256位宽的AVX指令集。
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j += 8) {  // 一次处理8个浮点数
            __m256 c = _mm256_load_ps(&C[i*m + j]);  // C[i][j] 到 C[i][j+7]
            for (int k = 0; k < p; k++) {
                __m256 a = _mm256_broadcast_ss(&A[i*p + k]);  // A[i][k]
                __m256 b = _mm256_load_ps(&B[k*m + j]);       // B[k][j] 到 B[k][j+7]
                c = _mm256_add_ps(c, _mm256_mul_ps(a, b));
            }
            _mm256_store_ps(&C[i*m + j], c);  // 将结果存回C[i][j] 到 C[i][j+7]
        }
    }
}
```

上述代码展示了AVX指令集的基本使用方式来执行矩阵乘法。_mm256_load_ps 函数加载8个连续的浮点数到一个256位的寄存器，_mm256_broadcast_ss 将单个浮点数复制到所有元素中，_mm256_mul_ps 和 _mm256_add_ps 分别执行元素级别的乘法和加法，最后 _mm256_store_ps 将寄存器中的值存回内存。

面向特定领域的体系结构(Domain Specific Architectures，DSAs)

- 通过根据领域特征调整体系结构，实现更高的效率
  - 不同于严格的ASIC，不需要对于每一个应用程序进行调整，而是对一个应用领域进行调整
  - 需要比通用处理器(general purpose)更多的特定领域知识(domain-specific knowledge)
- 示例：
  - 用于机器学习的神经网络处理器(Neural network processors for machine learning)
  - 用于图形、虚拟现实的GPU(GPUs for graphics, virtual reality)
  - 可编程网络交换机和接口(Programmable network switches and interfaces)

为什么DSAs 能够胜出？

- 针对特定域更有效的并行性：
    - SIMD vs. MIMD
    - VLIW vs. 预测、乱序执行的架构
- 更有效地利用内存带宽
    - 用户控制 vs 缓存
- 消除不必要的准确性
    - 低精度FP(Floating Point)取代IEEE标准的浮点数
    - 32-64位整数到8-16位整数
- 面向特定领域编程语言

面向特定领域的语言(Domain Specific Languages)

- DSAs要求将高级别操作作为体系结构的目标
- 很难从C或类似Python的语言开始并恢复结构
- 需要矩阵、向量或稀疏矩阵运算
- 域特定语言指定以下操作：
  - OpenGL, TensorFlow, P4(Programming Protocol-Independent Packet Processors)
- 如果DSL程序保持体系结构的独立性，那么编译器将面临有趣的挑战
  - XLA，“XLA－TensorFlow, Compiled”, XLA Team, March 6, 2017

Research Opportunities 研究机遇

- 通用应用：
  - 使用编译器+硬件使Python像C一样运行
  - 在RISC上快速实现HLL(High Level Language, 高级语言编程)
- 特定领域的应用程序（更大的机会？）
  - 什么是适用于重要应用程序的正确的DSLs?
  - 新的DSLs和DSAs的代码设计
  - 用于优化匹配的高级编译技术:
    - 新领域：不是从C/Fortran中提取高级结构，而是匹配/优化
- 挑战：不要用短期ISA定制或微体系结构定制的妥协来损害DSLs

DSL/DSA总结

- 存在大量的机遇
- 然而，目前的计算机体系结构方法可能无法满足面向特定领域的语言（DSL）和面向特定领域的体系结构（DSA）的需求。因此，我们需要新的计算机体系结构方法。
- 计算机体系结构团队需要具备垂直整合能力，包括：
  - 应用能力
  - DSL（面向特定领域的语言）及相关编译器技术
  - 架构原理
  - 实现技术

开源架构

- 为什么开放源代码编译器和操作系统，而不是ISA（ Instruction Set Architecture, 指令集架构）？
- 如果有免费和开放的ISA供我们在所有方面使用，会怎样呢？

RISC-V的起源故事

- 有传言称，加州大学伯克利分校的研究使用x86和ARM？
  - 不可能，因为太复杂且存在知识产权问题
- 他们在2010年开始了一个“为期3个月的项目”，以开发自己的全新ISA（指令集架构）
  - Krste Asanovic, Andrew Waterman, Yunsup Lee, Dave Patterson
- 四年后，发布了基本用户规范

为什么外界人士对伯克利课程中RISC-V的变化提出质疑？

RISC-V有何不同之处？

- 简单
  - 比专有的指令集架构要小得多
  - x86和ARMv8手册有2500页，而RISC-V手册只有200页
- 从零开始的全新架构设计方法(Clean-slate design)
  - 从前辈们的错误中吸取教训
  - 避免微架构或依赖于特定技术
- 模块化的
  - 简化的标准基础ISA
  - 多个标准扩展
- 支持专业化
  - 预留了广阔的操作码空间
- 社区设计
  - 基础扩展和标准扩展都已完成
  - 通过可选扩展实现增长，而非逐步增加所需特性
- 扩展ISA的目的不同
  - RISC-V基金会出于技术和体系结构发展的目的扩展和维护ISA
  - 私营公司出于盈利的目的扩展和维护ISA

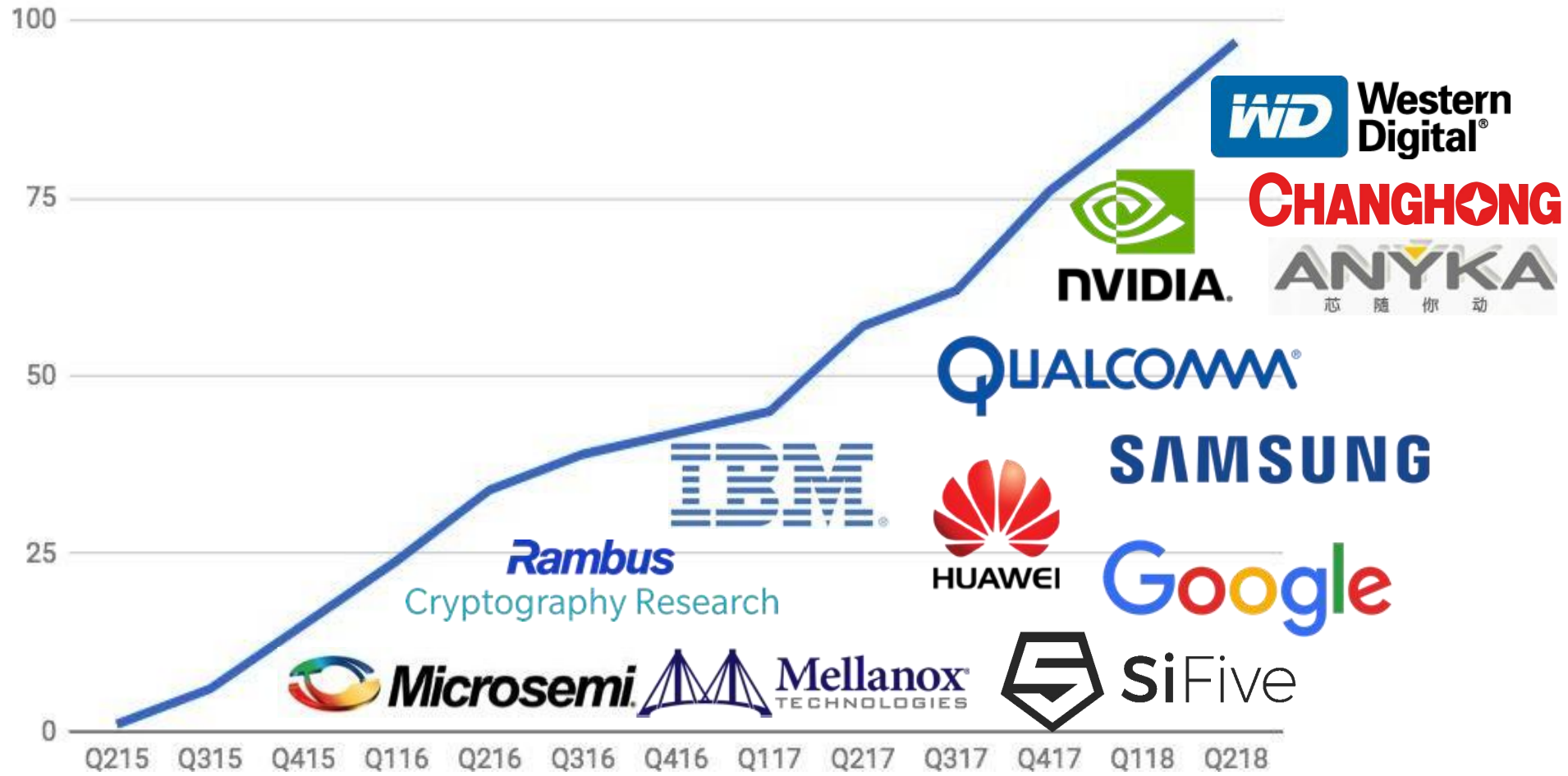
RISC-V基础加标准扩展

- 几种基本的整数ISA
  - RV32E, RV32I, RV64I
  - 基本指令集中的硬件指令少于50个（类似于RISC-I！*）
- 标准的RISC编码，采用固定的32位指令格式
- 由RISC-V基金会永久支持

- 标准扩展
  - M: 整数整除
  - A: 原子存储器操作
  - F/D: 单精度/双精度浮点数
  - C: 压缩指令
  - V: 面向DLP的向量扩展

“How close is RISC-V to RISC-I?” David Patterson, 9/19/17, ASPIRE Blog
“SIMD Instructions Considered Harmful.” David Patterson and Andrew Waterman, 9/18/17

2015年以来RISC-V基金会的成员

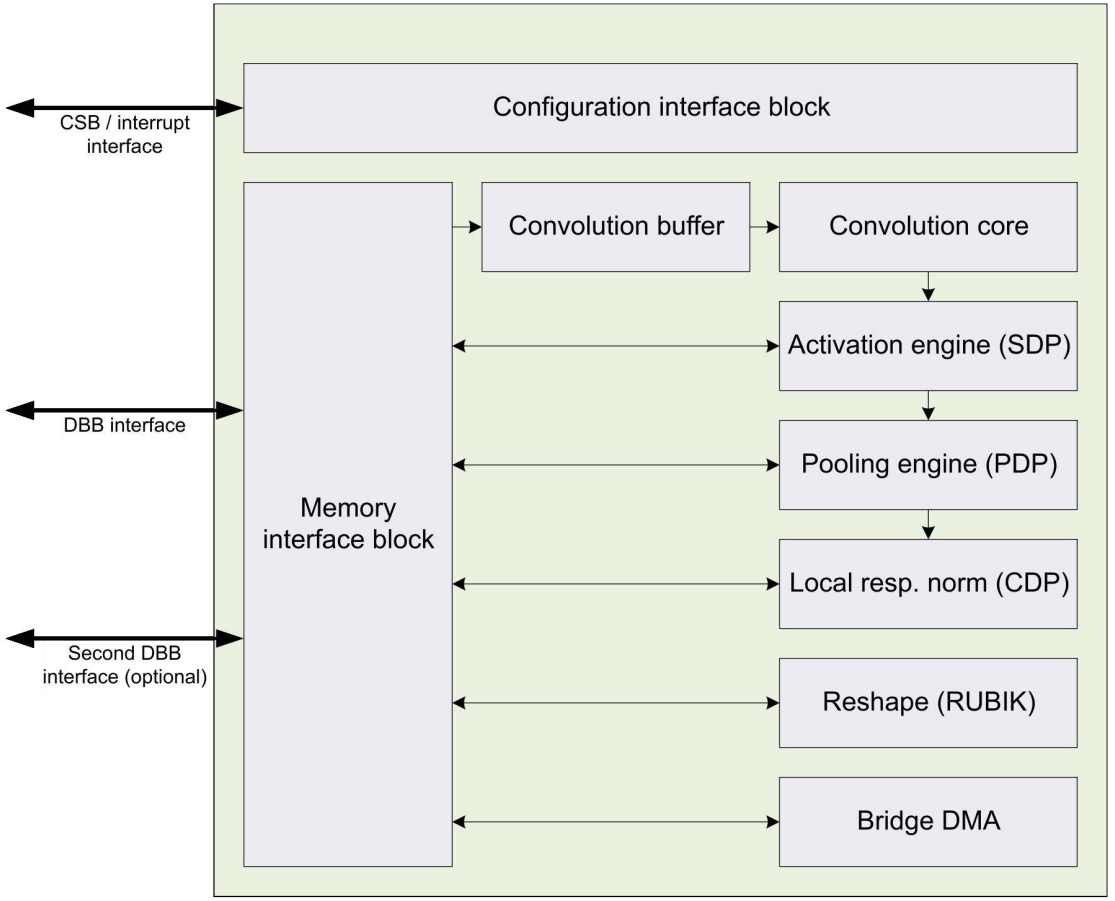


基础工作组（部分列表）


AMD
ANDES TECHNOLOGY
BAE SYSTEMS
Esperanto Technologies
ETH zürich
lowRISC
MEDIATEK
Microsemi
SiFive
Syntacore
Bit Manipulation
codasip
BAE SYSTEMS
bluespec
Microsemi
NVIDIA
SiFive
Syntacore
Berkeley Architecture Research
Compliance
SiFive
ANDES TECHNOLOGY
BAE SYSTEMS
bluespec
cortus
Google
NVIDIA
lowRISC
ROA LOGIC
Syntacore
VeriSilicon
ultraSoC
Debug
NVIDIA
IBM
PRINCETON UNIVERSITY
VeriSilicon
lowRISC
MIT CSAIL
SiFive
ANDES TECHNOLOGY
bluespec
BAE SYSTEMS
Western Digital
Syntacore
Memory Model
SiFive
ANDES TECHNOLOGY
BAE SYSTEMS
Esperanto Technologies
bluespec
NVIDIA
Western Digital
lowRISC
Berkeley Architecture Research
MIT CSAIL
Syntacore
Privileged Spec
SiFive
ANDES TECHNOLOGY
NVIDIA
Micron
Google
lowRISC
MEDIATEK
Microsemi
Esperanto Technologies
Syntacore
Berkeley Architecture Research
ETH zürich
Western Digital
BAE SYSTEMS
Vector
NVIDIA
MEDIATEK
HUAWEI
Microsemi
SiFive
Blockstream
Esperanto Technologies
BAE SYSTEMS
ANDES TECHNOLOGY
DRAPER
ultraSoC
AMD
NXP
Security
SiFive
ANDES TECHNOLOGY
Esperanto Technologies
Western Digital
Syntacore
lowRISC
Berkeley Architecture Research
BAE SYSTEMS
Base ISA/Opcode

NVDLA：一个开放的DSA和实现

- NVDLA: NVIDIA深度学习推理加速器
- 免费和开放：所有软件、硬件和文档都在GitHub上
- 可扩展的、可配置的设计
  - 每个模块可以独立操作或通过流水线方式绕过内存
  - 数据类型可配置： int8、int16、fp16
  - 可配置的2D MAC阵列
  - 尺寸按比例缩放6倍（0.5 – 3mm2），功耗按比例缩放15倍（20 – 300 mW）
- 将RISC-V核心作为主机



安全和开放架构

- 安全社区喜欢可验证（无后门*）、可修改、自由和开放的架构和实现
- 同样重要的是执行架构实验的人员和组织的数量
- FPGA的可塑性 + 开源RISC-V实现和软件 ⇒ 新颖架构可以在线部署、评估和迭代，时间周期为几周而不是几年
- 通过架构师和安全专家的硬件/软件协同设计，RISC-V有望成为安全性的典范

Sturton C, Hicks M, Wagner D, et al. Defeating UCI: Building stealthy and malicious hardware[C]//2011 IEEE symposium on security and privacy. IEEE, 2011: 64-77.

开放式体系结构总结

- 自由/开放式体系结构→ 没有所有权锁定(proprietary lock-in)，没有合同限制，任何人都可以使用/购买/销售
- 通常较简单，不受市场推动(marketing driven)（有助于验证和安全性）> 区域/功耗/性能方面低端和高端产品接近
- 容易且自由地可扩展（支持面向特定领域的体系架构DSAs ）
- 越来越多的组织设计处理器（开源）
- ⇒ 更快的创新，更具竞争力的市场
- 将成为安全专家的主要实验工具？

开放式体系结构目标(Open Architecture Goal)

- 为所有计算设备创建行业标准的开放式指令集架构 "Linux for processors"

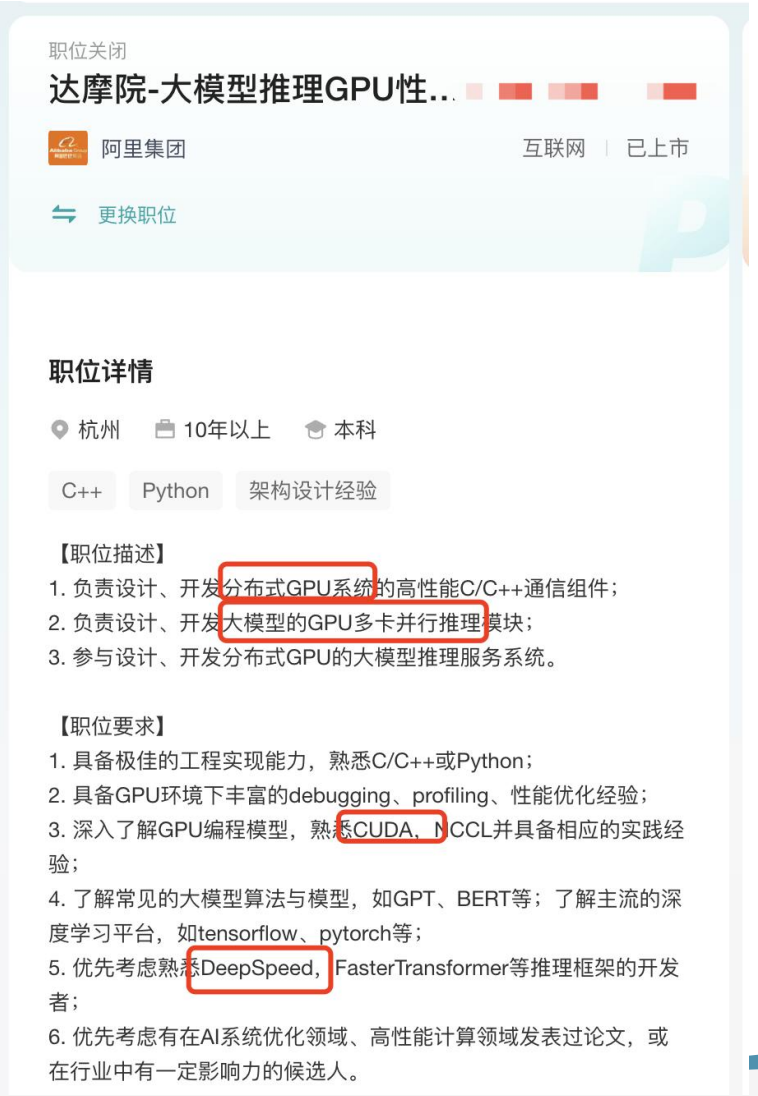
就业前景

- Last but not the least，学习这门课有什么最直接的好处呢？

招聘要求

- 本科以上学历，通信、电子、计算机相关专业毕业，工作经验三年以上者优先。
- 扎实的编程能力（C++/C、Go、Rust、Python等）。
- 了解最新AI问题特点，有搜推广、NLP、CV、自动驾驶等任意一个场景下AI模型的训练、部署的从业经验。
- 对最新的AI软件栈有比较深入理解，熟系一种AI模型部署或训练解决方案，如TensoRT、vLLM、TGI、ONNX、Megatron-LM，DeepSpeed等。
- 有并行计算、高性能计算、分布式计算任意一种技能者优先。
- 熟系一种CUDA、OpenAITriton、TVM、MLIR等异构硬件编程语言或编程工具。
- 了解计算机体系结构基本原理，有NVIDIA、Intel、AMD、国产AI芯片中一种高性能处理器开发经验者优先。







授课教师：王强
邮箱：qiang.wang@hit.edu.cn

谢谢

Thank You